ARCHITECTURE OF ML-ENABLED SYSTEMS

ML Architecture.

From prediction to production.
A bank customer-churn case.

A model becomes useful inside a system.

Data and feature pipelines. Serving and monitoring. Controlled change and quality.

PRESENTERS

Ricardo · Tomás · Simon · Luca · Matheus

Based on the group outline, assigned readings, and shared study notes.

BLOCK 1 / FOUNDATIONS

Two blocks, one shared bank case

Follow the group's topic outline; use the case to connect each architectural decision.

Block	Topics	Presenter
1 · Model to system	Foundations and dual lifecycle	Ricardo
	Deployment and serving	Tomás
	Monitoring and degradation	Simon
	Design decisions and technical debt	Luca
2 · Architecture & quality	Architectural tactics	TBD
	DevOps / MLOps	Matheus
	Security and privacy	Simon + Matheus
	Quality checklist and tools	TBD

Key concepts

- **Block 1:** lifecycle, serving, monitoring, and design decisions.
- **Block 2:** tactics, MLOps, security, privacy, and quality.
- **Concept first:** explain the mechanism, then apply it to churn.
- **Named handoffs:** each section identifies its presenter; unassigned topics remain TBD.

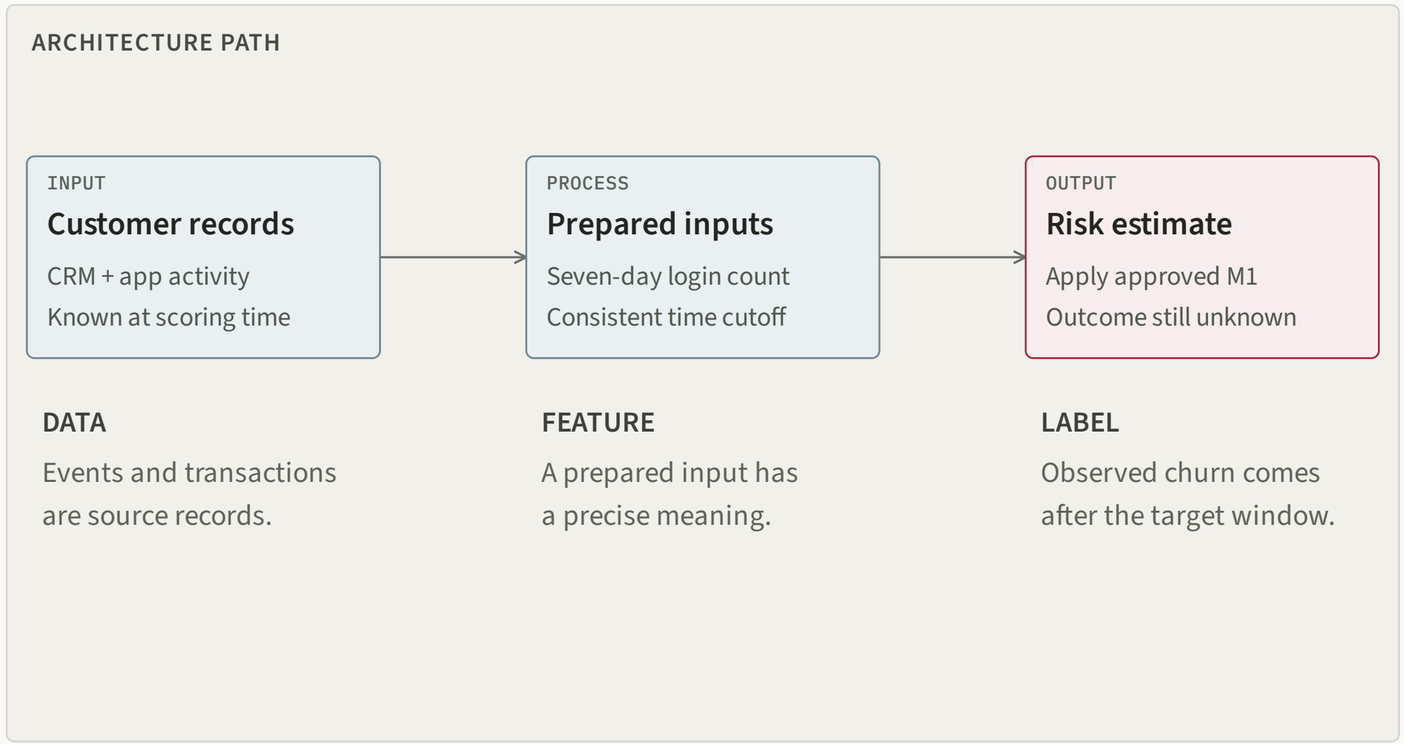
Each architectural decision connects predictive behavior to a system responsibility.

BANK EXAMPLE — The same source → feature → model → score → policy path returns throughout the lecture.

BLOCK 1 / FOUNDATIONS

The bank case: features, labels, and predictions

A few ML terms connect familiar application data to a learned risk estimate.



Ana / C042 and all values are illustrative. A feature value does not prove intent.

Key concepts

- **Feature:** a prepared input, such as logins during the previous seven days.
- **Label:** the observed churn outcome used for training and later evaluation.
- **Prediction:** an estimate produced before the customer's future outcome is known.
- **Training vs. inference:** learn from earlier examples; apply the approved model to current inputs.

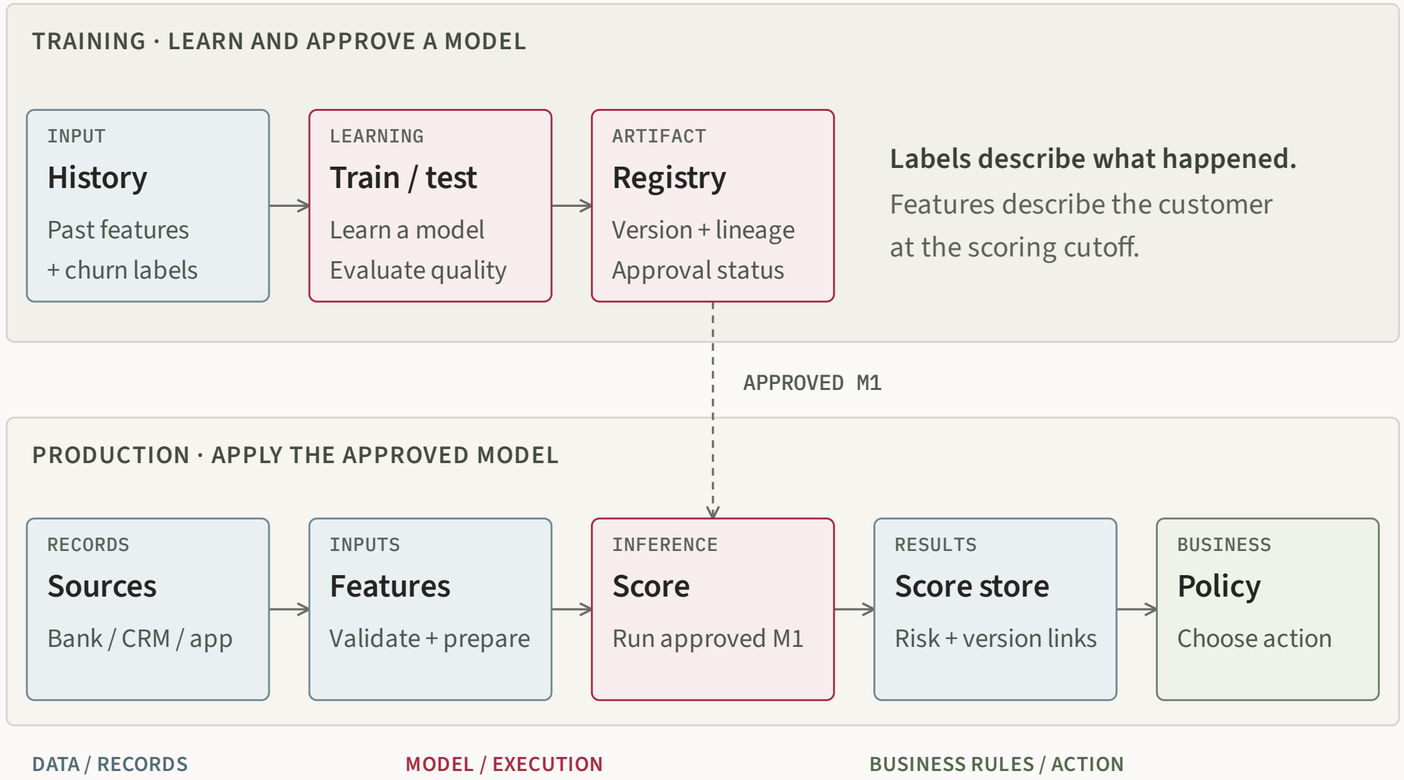
Lower activity is evidence for a model input, not proof that a customer will leave.

BANK EXAMPLE — Ana / C042: logins fall from 7 to 1; the illustrative score is 0.70. Her next-30-day outcome is unknown.

BLOCK 1 / FOUNDATIONS

From a trained model to an ML system

Architecture connects learning, production data, operational controls, and business decisions.



Key concepts

- **Training** uses historical features and labels to build and evaluate candidates.
- **Production** validates current data and applies an approved, compatible model.
- **Stored scores** link predictions to customer, time, input evidence, and versions.
- **Business policy** determines the permitted action; it remains separate from the score.

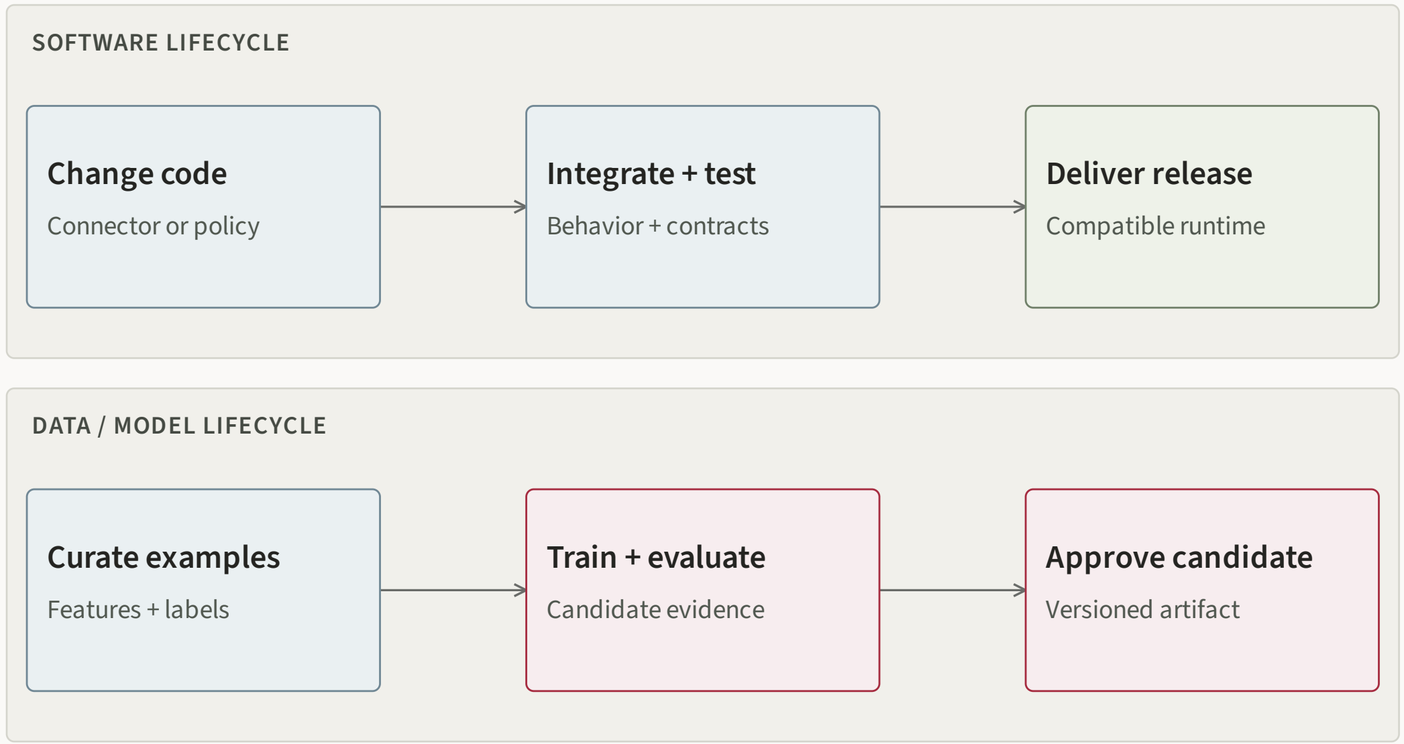
Keep the learning path and production path distinct, but traceable.

BANK EXAMPLE The bank computes a monthly batch of churn scores, then applies campaign eligibility and contact rules.

BLOCK 1 / FOUNDATIONS

Two lifecycles, one compatible system

Code correctness and predictive usefulness depend on different feedback and release cycles.



Coordinate code, data meaning, model, and configuration; neither path is sufficient alone.

Key concepts

- **Software changes** can be checked through code, contract, and integration tests.
- **Model development** also needs representative data, suitable labels, and evaluation.
- **Feedback differs:** execution checks may be immediate; churn labels arrive later.
- **Co-versioning** links changes across code, feature meaning, model, and configuration.

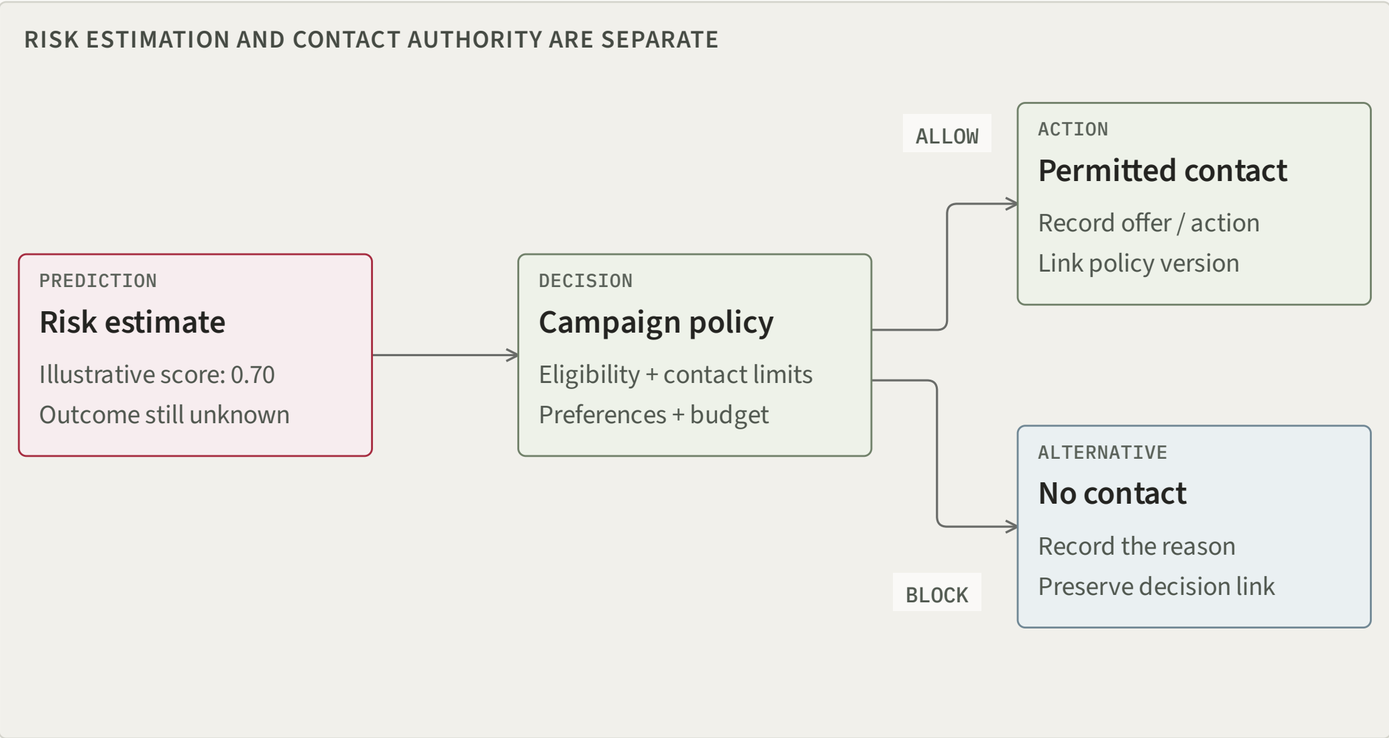
A passing software test does not establish future predictive quality.

BANK EXAMPLE A CRM connector can change today while M1 stays fixed; its changed inputs can still alter predictions.

BLOCK 1 / FOUNDATIONS

Prediction and business action are separate

A model estimates risk. Business rules decide whether and how the bank may act.



A high risk estimate does not bypass the policy. No completed contact or outcome is asserted here.

Key concepts

- **Score:** estimated churn risk, not a guaranteed outcome.
- **Policy:** preferences, eligibility, contact limits, and campaign budget.
- **Action:** an offer, another response, or no contact at all.
- **Separate evaluation:** predictive quality and the benefit of an offer are different questions.

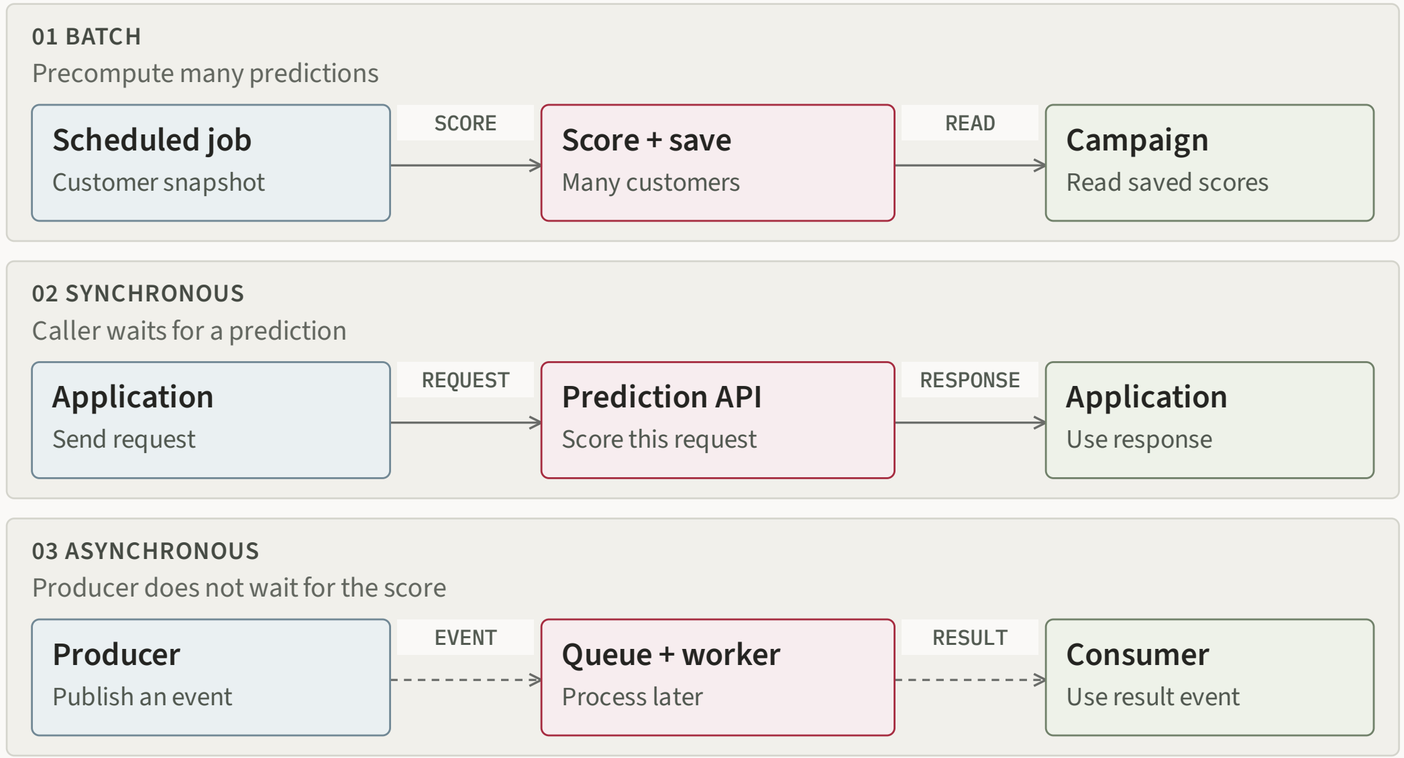
A policy change should not require retraining the prediction model.

BANK EXAMPLE — Ana may have a high risk score and still receive no offer because contact preferences or policy prohibit it.

BLOCK 1 / DEPLOYMENT & SERVING

Serving patterns: when is a prediction needed?

Select the interaction pattern from the decision's timing and failure behavior.



Key concepts

- **Batch:** compute many predictions; consumers read stored results later.
- **Synchronous:** the caller waits for a result during its request.
- **Asynchronous:** an event producer and consumer are decoupled through messaging.
- **Edge:** a deployment location, independent of prediction timing.

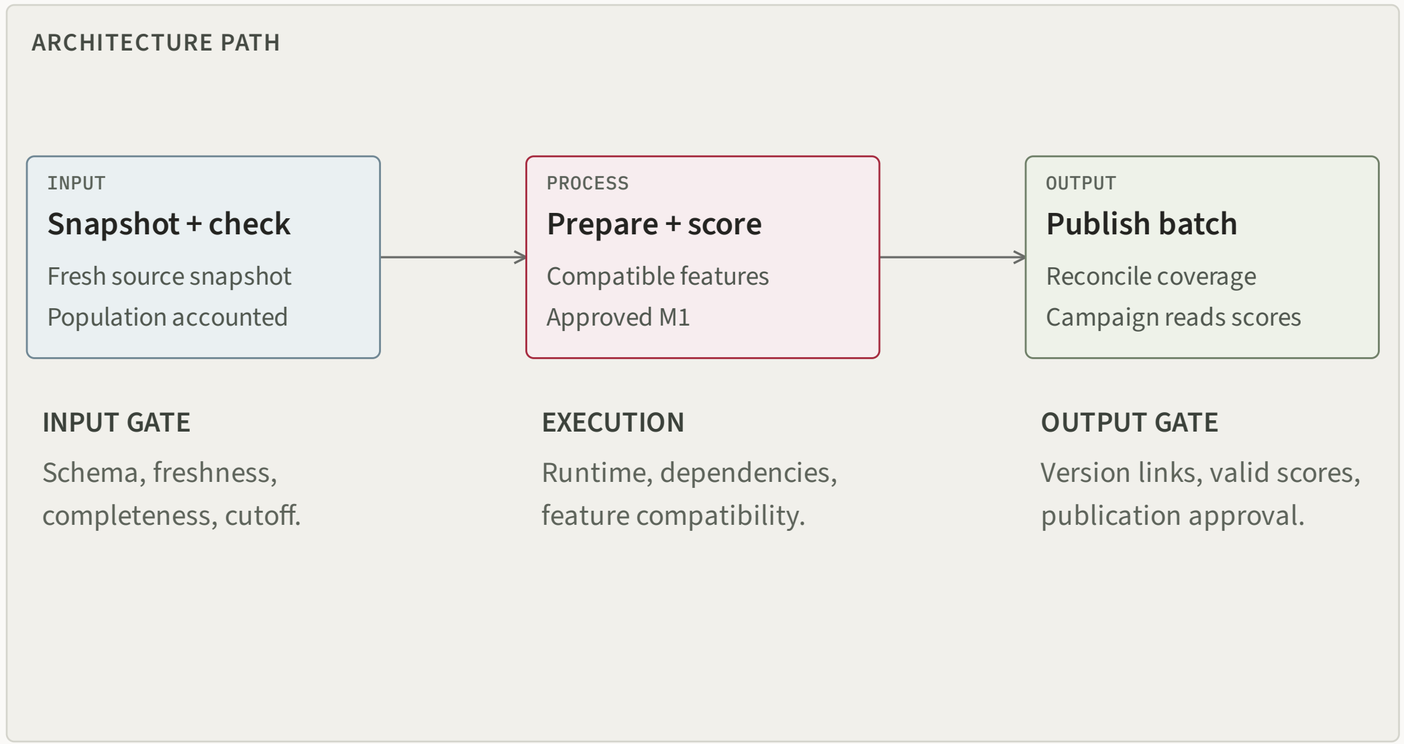
Timing, location, and release strategy are separate choices.

BANK EXAMPLE — The churn campaign can use precomputed scores. An immediate fraud check motivates a different timing need.

BLOCK 1 / DEPLOYMENT & SERVING

Batch serving: validate, score, publish

A complete validated batch must be available before its business deadline.



Account for the expected population; a successful final task alone is not a complete batch.

Key concepts

- **Snapshot:** define population, cutoff, and required source freshness.
- **Validate:** reject incompatible inputs before scoring.
- **Reconcile:** account for written, rejected, and withheld records.
- **Publish:** expose a validated batch under a defined consumer contract.

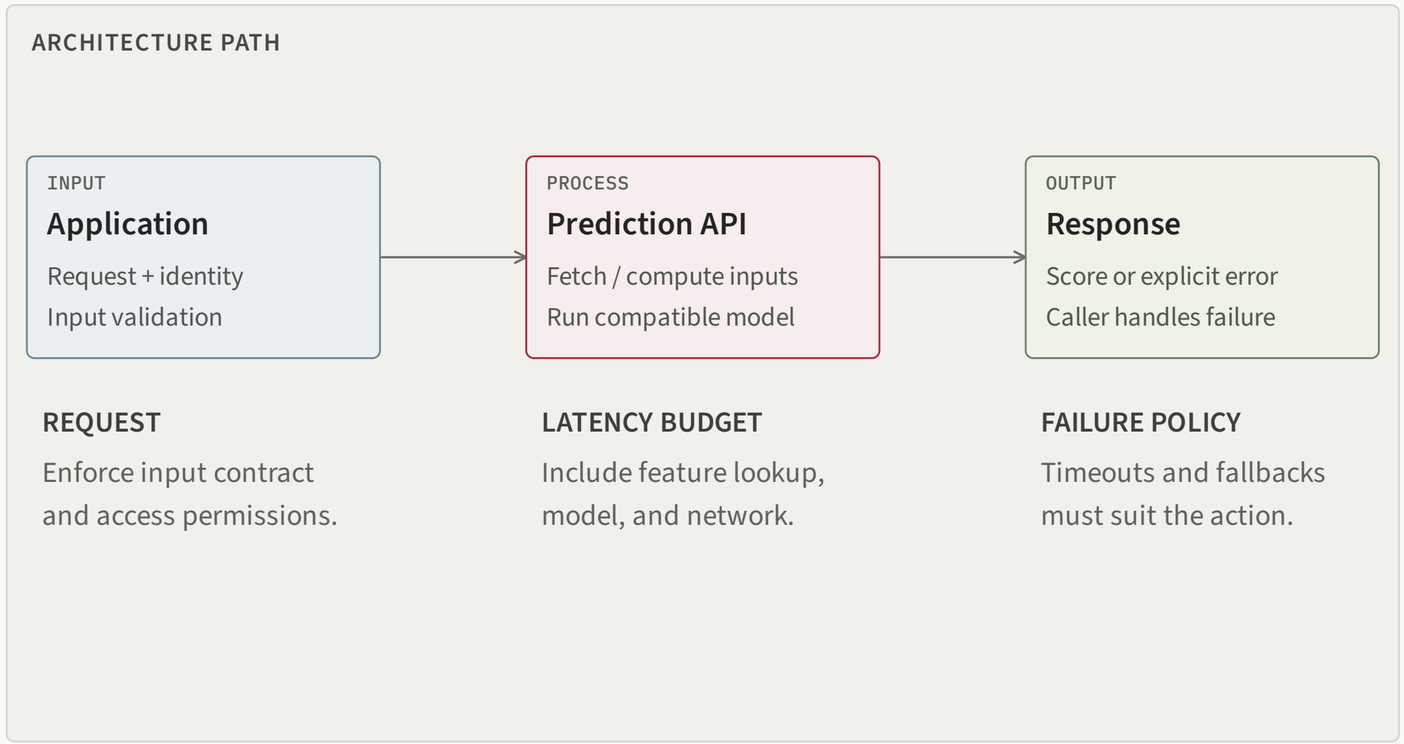
Optimize the whole batch deadline, not only per-record prediction speed.

BANK EXAMPLE Monthly churn scores are saved for the campaign. A real numerical deadline must be agreed with campaign owners.

BLOCK 1 / DEPLOYMENT & SERVING

Synchronous serving: the caller waits

Prediction becomes part of the application's response path.



This is a pattern comparison, not a new online component in the monthly churn architecture.

Key concepts

- **Response time** includes lookup, preparation, inference, and network work.
- **Availability** now affects the waiting application's request.
- **Failure behavior** needs explicit timeout, error, and fallback policies.
- **Consistency** still requires compatible feature meanings and model versions.

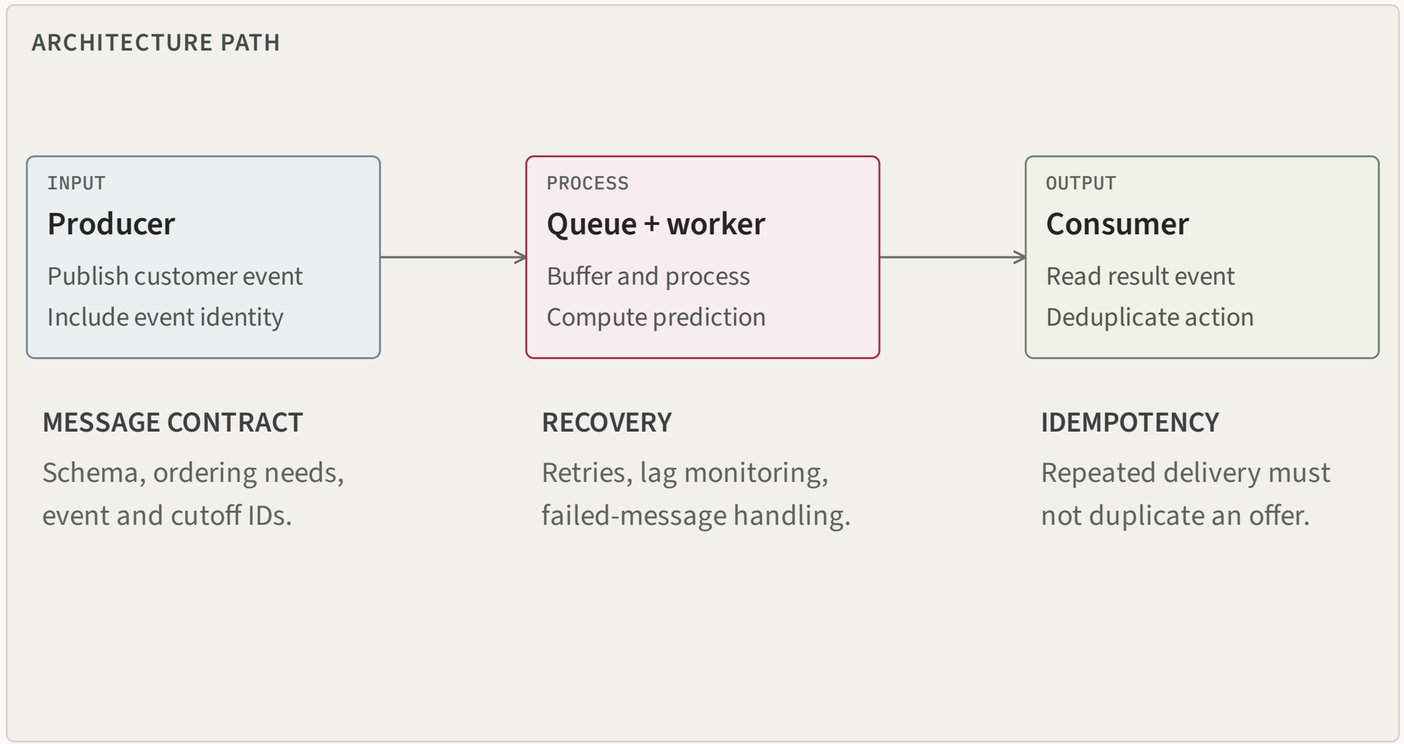
A fast model does not guarantee a fast end-to-end service.

BANK EXAMPLE An immediate fraud check can justify this pattern. The monthly churn campaign does not require it.

BLOCK 1 / DEPLOYMENT & SERVING

Asynchronous serving: decouple the work

Messages separate production, processing, and consumption of a prediction.



Asynchronous interaction can still have a strict deadline. Delivery guarantees require an explicit design.

Key concepts

- **Buffering** absorbs bursts and supports independent processing.
- **Lag** and queue health become operational concerns.
- **Retries** require failure handling and idempotent processing.
- **Contracts** define message identity, timing, schema, and result interpretation.

Decoupling adds messaging responsibilities; it does not remove timing requirements.

BANK EXAMPLE If a future campaign consumes prediction events, a repeated message must not cause duplicate customer contacts.

BLOCK 1 / DEPLOYMENT & SERVING

Deployment location is a separate decision

Central, embedded, and edge deployments change operations and constraints.

Location	Typical benefit	Architecture cost
Central service / job	Shared governance and compute	Connectivity and centralized availability
Embedded application	Local integration and low call overhead	Coupled application/model releases
Edge device	Local response and offline operation	Resource limits, rollout and stale versions

Key concepts

- **Central:** simplify shared operation and governance.
- **Embedded:** deploy the model with its host application.
- **Edge:** account for constrained hardware and intermittent connectivity.
- **Choose independently:** location does not uniquely determine batch or request timing.

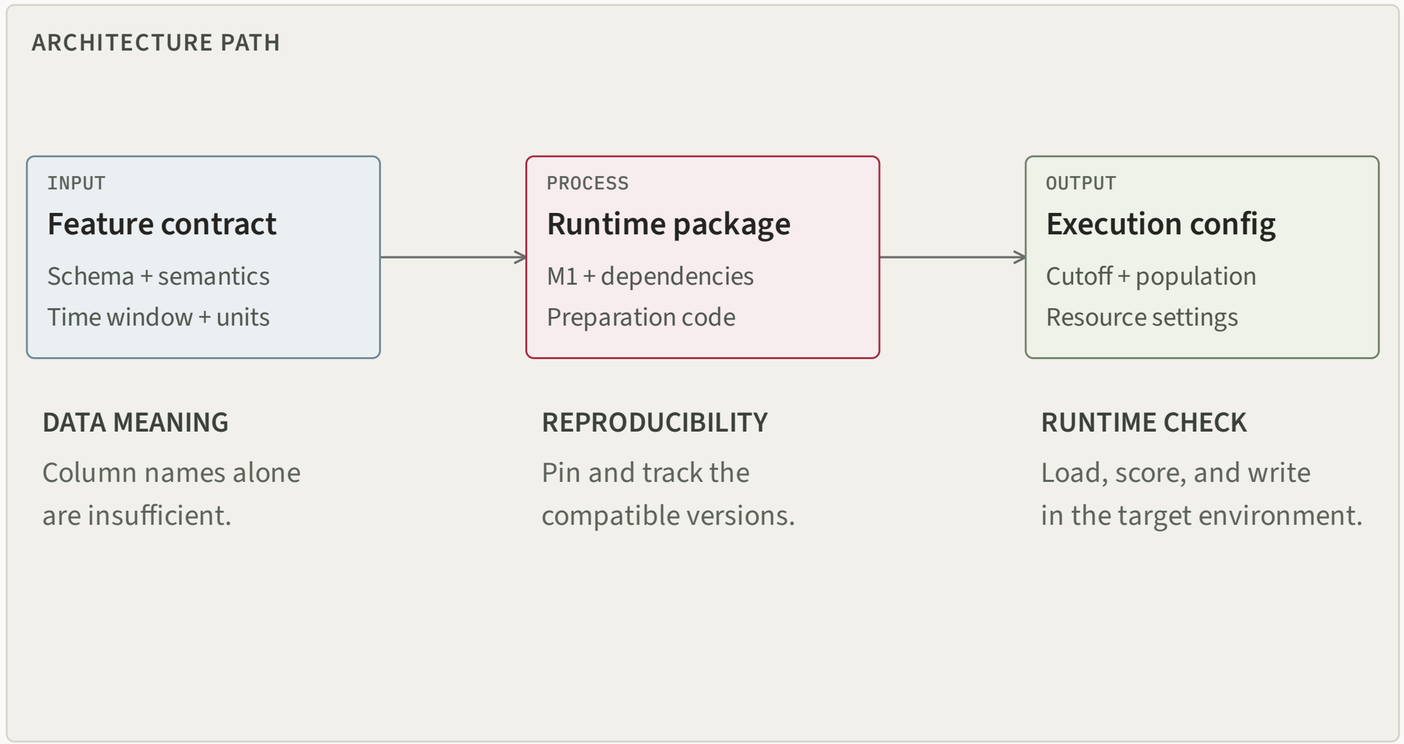
Architecture follows the operational context.

BANK EXAMPLE Keep churn centrally governed. Device-local inference is a contrast, not an extra subsystem to implement here.

BLOCK 1 / DEPLOYMENT & SERVING

Package the whole compatible set

A deployable release contains more than the learned model file.



Containerization packages execution; it does not itself establish predictive quality or release approval.

Key concepts

- **Model artifact:** the learned parameters and required format.
- **Runtime:** dependencies, preparation code, and execution environment.
- **Contract:** feature meaning, types, missingness, and temporal cutoff.
- **Configuration:** scoring population and execution settings.

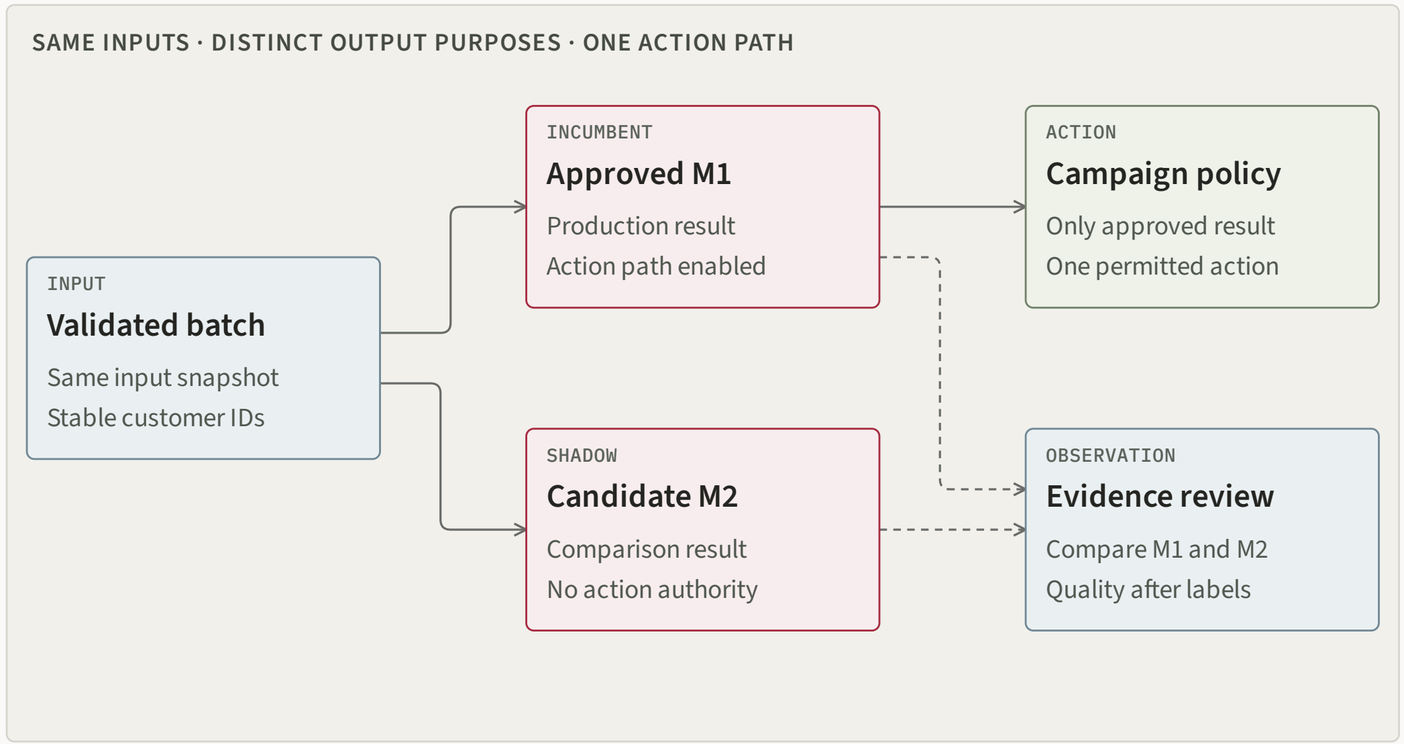
A package that loads successfully still needs release and quality evidence.

BANK EXAMPLE — The bank packages M1 with the exact feature interpretation needed to score the monthly customer snapshot.

BLOCK 1 / DEPLOYMENT & SERVING

Shadow deployment: observe without acting

A candidate scores real inputs while the approved model remains the action source.



The candidate has no path to campaign action. Comparison records preserve model and input identities.

Key concepts

- **Run in parallel** against comparable production inputs.
- **Record identity:** distinguish M1 and M2 outputs explicitly.
- **Compare** operational behavior and score differences first.
- **Wait for labels** to compare future churn quality.

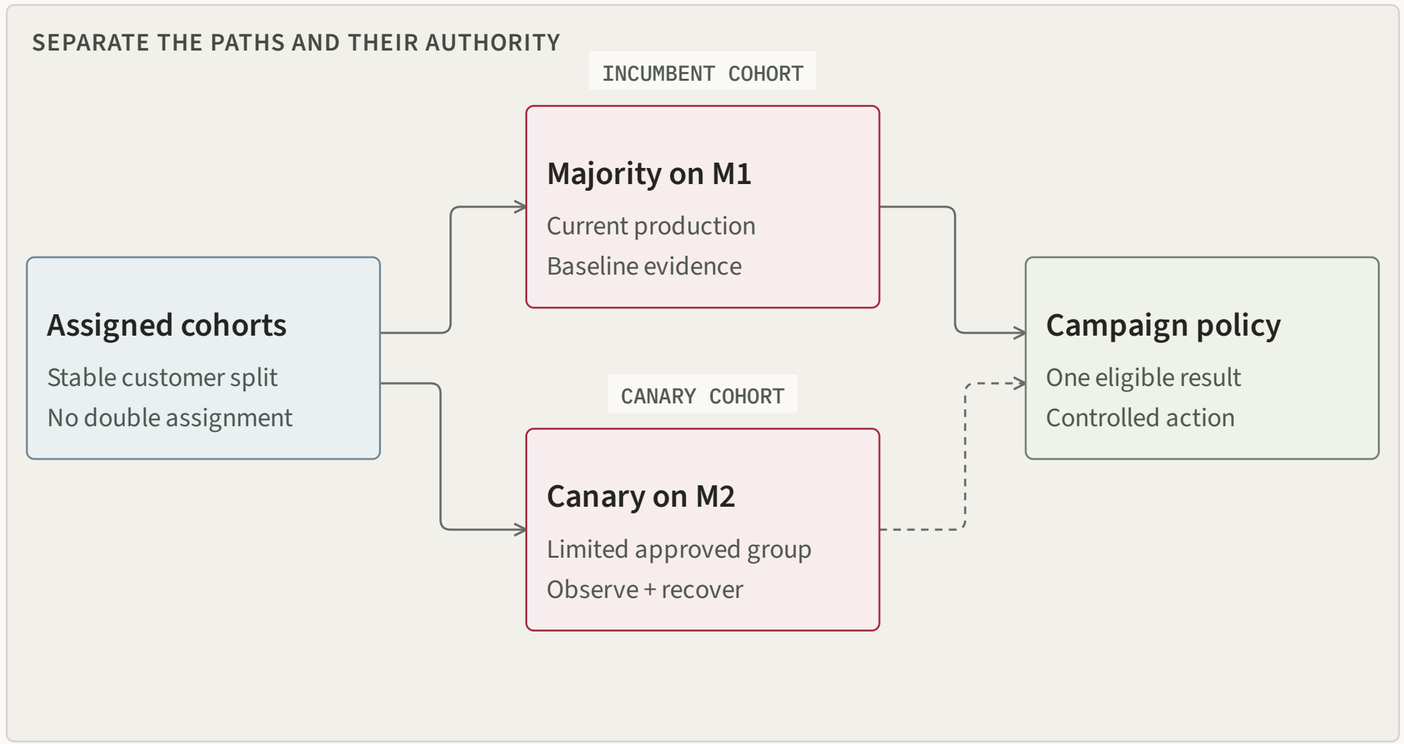
Shadow checks observation, not the business effect of acting on the candidate.

BANK EXAMPLE Only the M1 record is eligible for campaign use; M2’s shadow score cannot create a second offer.

BLOCK 1 / DEPLOYMENT & SERVING

Canary deployment: limit exposure

An approved limited cohort uses the candidate before broader rollout.



The exact cohort size and acceptance gates are design choices; no rollout percentage is prescribed here.

Key concepts

- **Restrict exposure** to a defined cohort and scope.
- **Observe** execution, compatibility, coverage, and policy behavior.
- **Define stop rules** and a rehearsed recovery path.
- **Separate evidence:** short operational success cannot prove 30-day predictive quality.

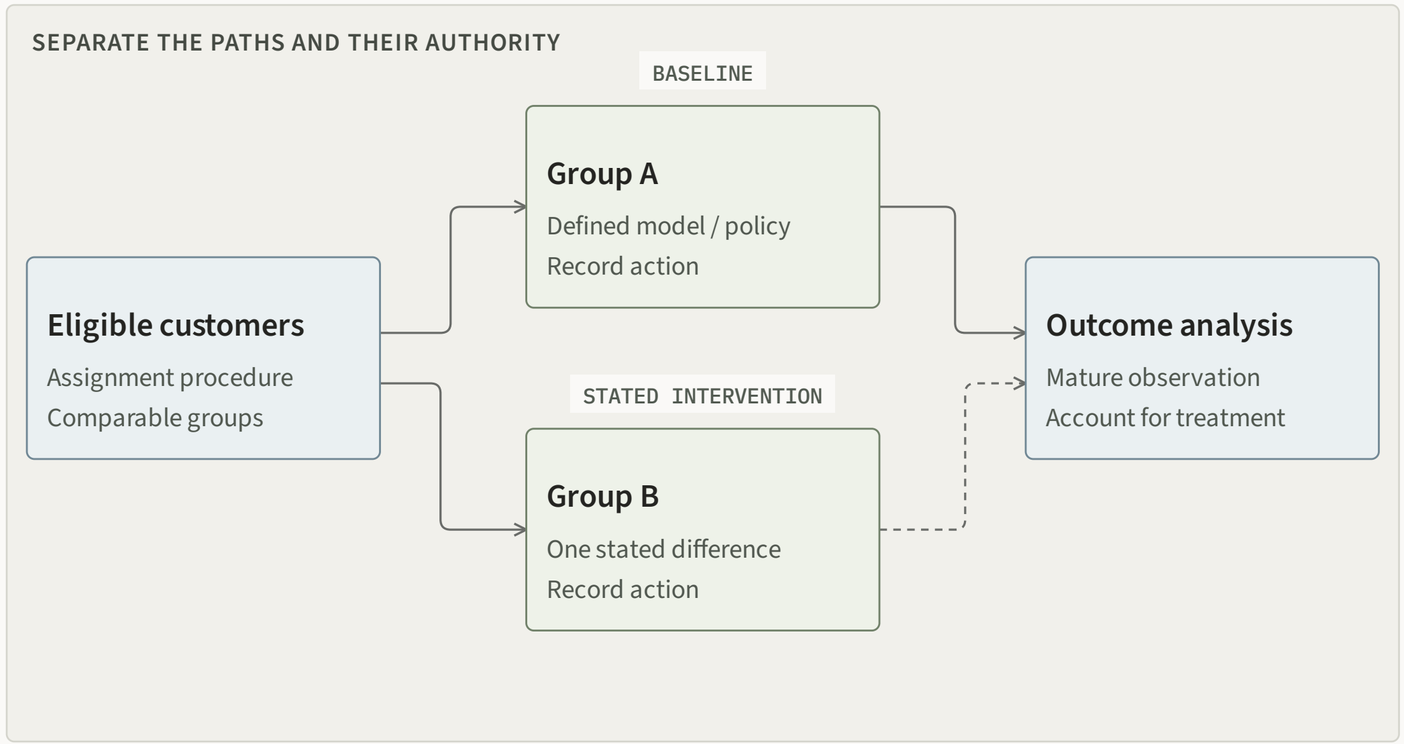
A canary controls risk; it is not automatically a valid business experiment.

BANK EXAMPLE — The bank may authorize a small cohort to use M2 while retaining M1 for the remainder.

BLOCK 1 / DEPLOYMENT & SERVING

A/B testing: define the intervention

Assigned groups and a stated outcome make the comparison interpretable.



A model comparison and a retention-treatment experiment answer different questions.

Key concepts

- **State what changes:** model, decision policy, or retention treatment.
- **Assign comparable groups** under an appropriate experiment design.
- **Define outcomes** and the observation window before interpreting results.
- **Avoid confounding:** changing model and offer together obscures their separate effects.

Prediction accuracy and the effect of contacting a customer are different targets.

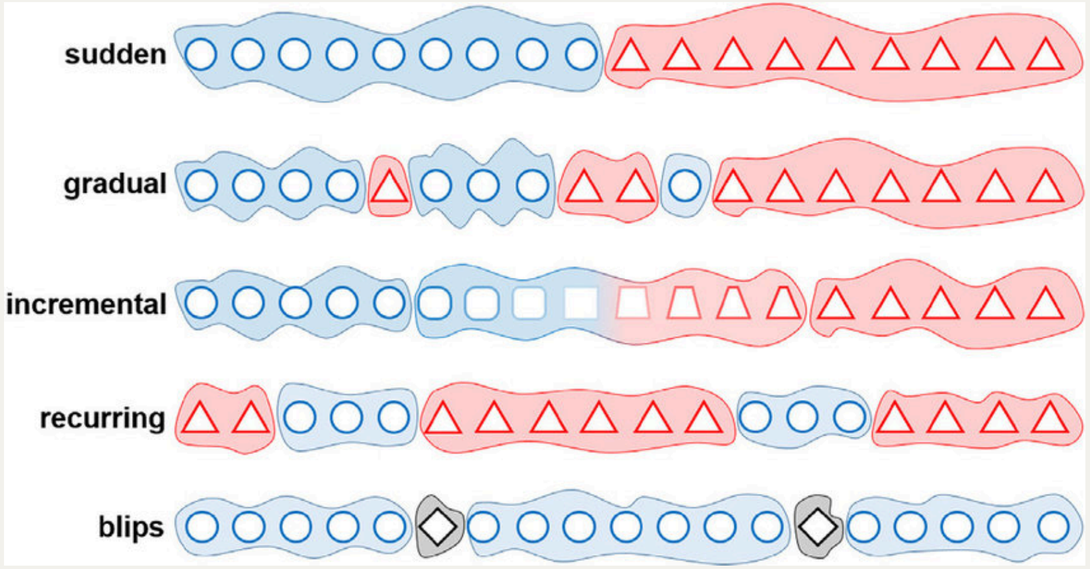
BANK EXAMPLE

Compare two offer policies separately from asking whether M2 ranks churn risk better than M1.

BLOCK 1 / MONITORING & DEGRADATION

How the world changes over time

Sudden, gradual, incremental, and recurring changes are different time patterns—not different input features.



A fee change takes effect overnight.

Customers adopt a new app over several weeks.

Average balances drift up slowly over months.

Holiday shopping spikes return every December.

A single unusual day, like Christmas Eve.

Krawczyk and Cano (2018), Figure 2.

BANK EXAMPLE Bank illustrations: a pricing change can be sudden; staggered adoption gradual; evolving habits incremental; seasonal activity recurring.

BLOCK 1 / MONITORING & DEGRADATION

Monitoring: observe more than model accuracy

Inputs, system execution, predictions, and business actions need different evidence.

Layer	Evidence	Response owner
Inputs	Freshness, missingness, distributions	Source / feature owners
System	Completion, latency, errors, coverage	Operations
Model	Scores and mature-label quality	ML reviewer
Action	Eligibility, contacts, outcomes	Campaign owner

Key concepts

- **Inputs:** are the right records available and meaningful?
- **System:** did the pipeline complete correctly and on time?
- **Model:** are predictions useful on the current population?
- **Business:** are permitted actions and outcomes being tracked?

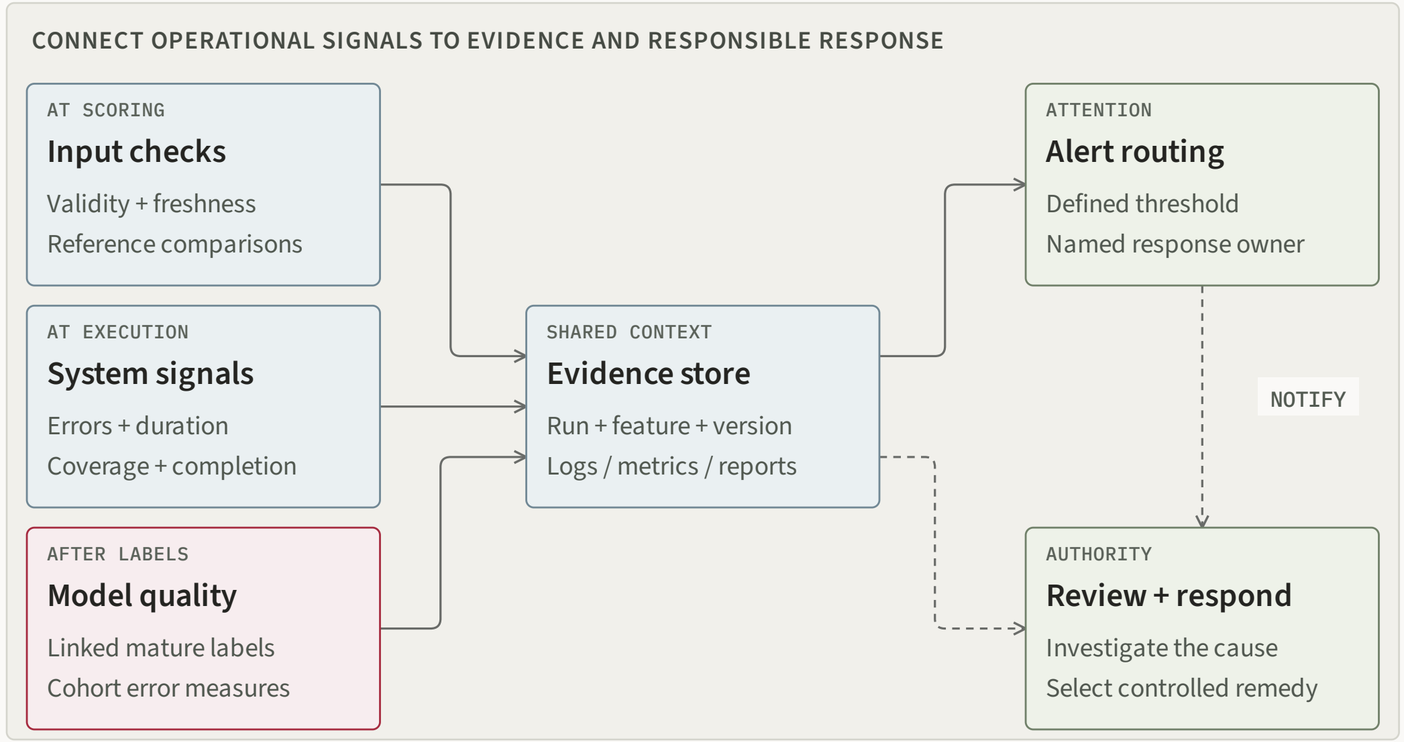
Observability connects evidence to investigation; dashboards alone do not decide the remedy.

BANK EXAMPLE A healthy churn job can still use a shifted population. A useful score can still lead to an inappropriate contact.

BLOCK 1 / MONITORING & DEGRADATION

Monitoring architecture: from signal to response

Collect linked evidence and route it to an owner who can investigate and act.



Monitoring can request a review. It cannot approve a candidate or silently promote a replacement.

Key concepts

- **Signals:** input, execution, and predictive evidence arrive at different times.
- **Context:** link observations to run, feature, model, and cohort identity.
- **Routing:** an alert has a defined recipient and response policy.
- **Authority:** investigation and release approval remain separate.

A useful monitoring design connects evidence, ownership, and a controlled response.

BANK EXAMPLE A CRM missingness alert links to B017 and reaches source / operations owners; it does not promote M2.

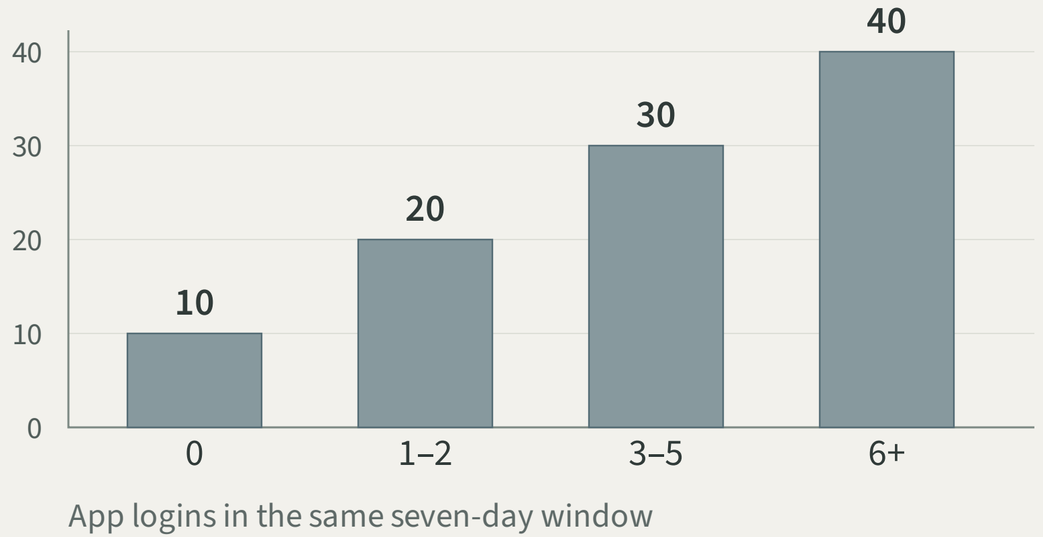
BLOCK 1 / MONITORING & DEGRADATION

Data drift: a different customer mix

Compare the same input definition across a reference cohort and the current cohort.

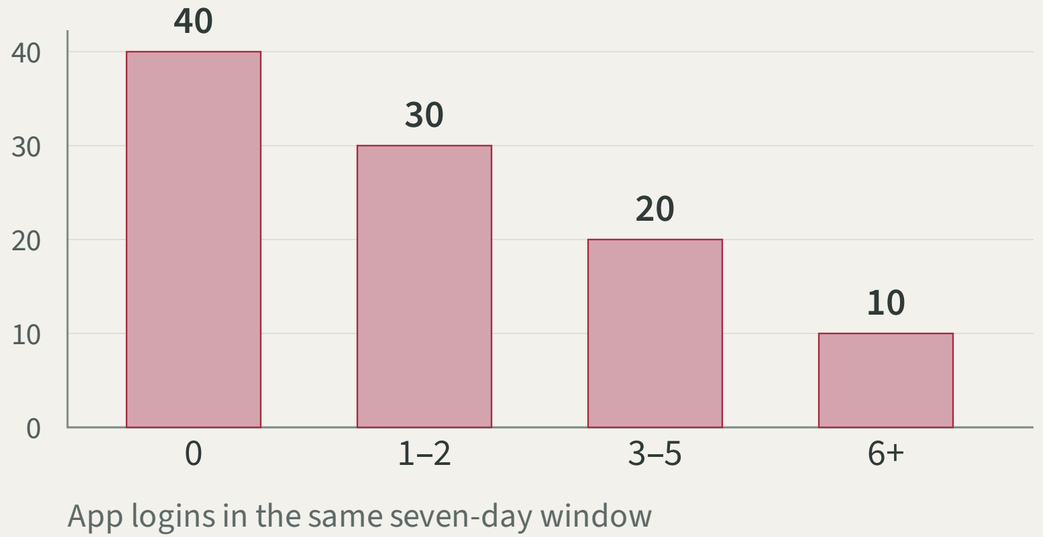
REFERENCE COHORT

Customers per 100



CURRENT COHORT

Customers per 100



WHAT CHANGED The mix of customer inputs. Churn labels are not needed to see this shift.

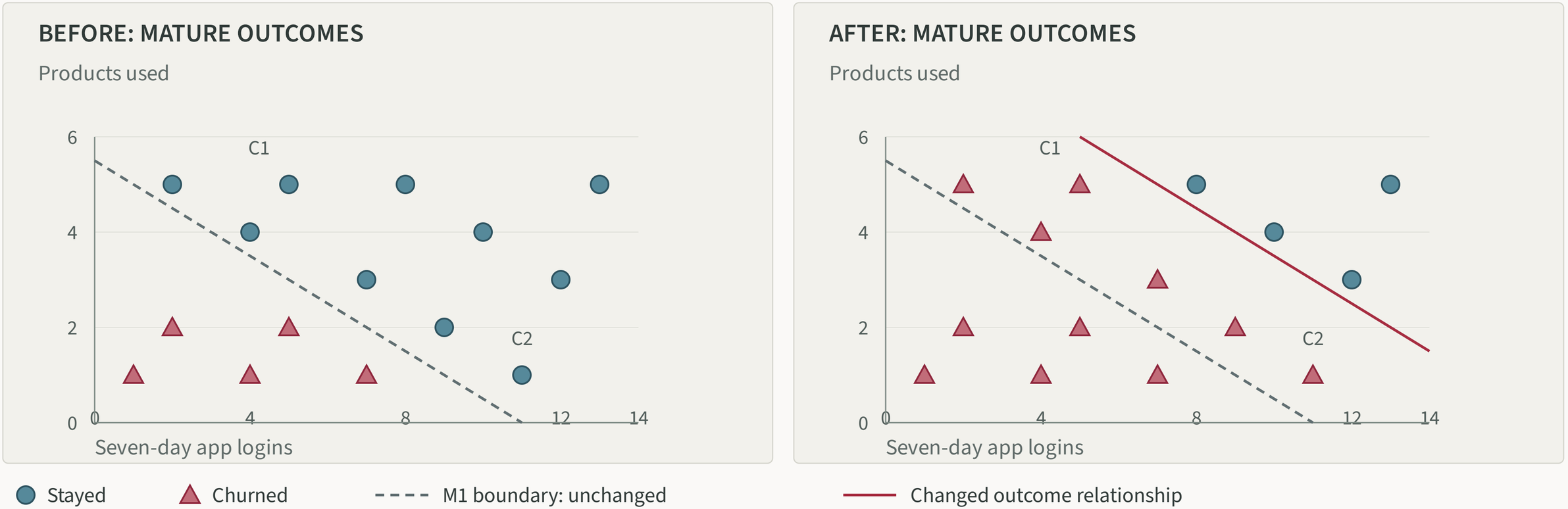
Invented cohorts: 100 customers each. Identical bins and feature meaning. Input drift alone does not establish accuracy loss.

BANK EXAMPLE Both diagrams count 100 fictional customers using the same seven-day login bins. Only the customer mix changes.

BLOCK 1 / MONITORING & DEGRADATION

Concept drift: a different relationship

The same inputs can have different outcomes while the deployed model keeps its earlier boundary.



Same 15 input positions in both panels. Shapes show invented mature outcomes; the solid line is not an automatically retrained model.

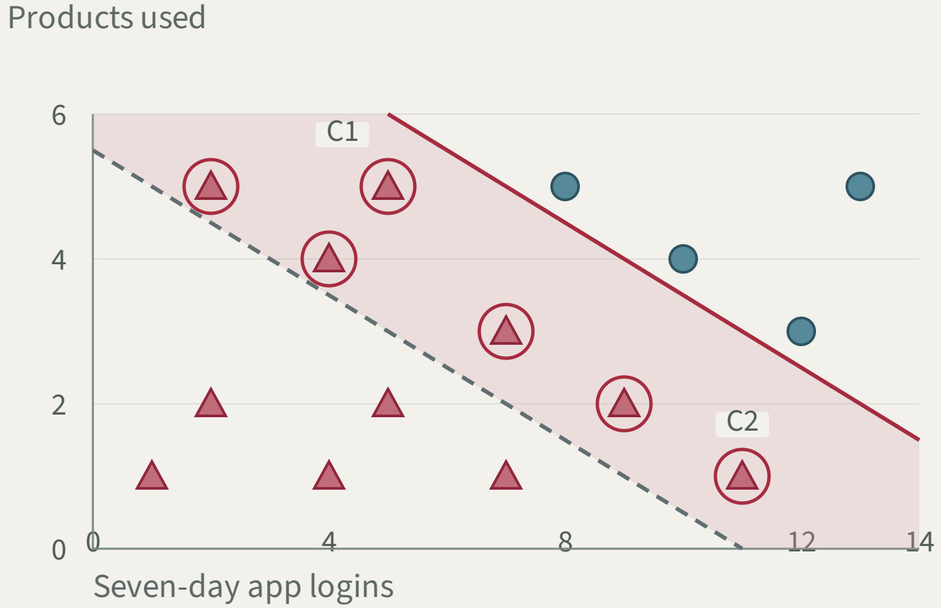
BANK EXAMPLE The two panels use the same 15 fictional feature positions. After the hypothetical change, some previously staying customers churn.

BLOCK 1 / MONITORING & DEGRADATION

Concept drift: how M1 starts making errors

A fixed model can miss churners because the world changed—not because its software stopped running.

AFTER: MATURE OUTCOMES



Rings identify missed churners. Shading marks the changed region. M1 is still unchanged.

WHY M1 NOW MISSES CUSTOMERS

1 The input definitions did not change.

2 M1 still uses its earlier boundary.

3 Observed churn now follows a different pattern.

6 customers change outcome in this toy example.

They sit between the old and new boundaries.

C1 · 5 logins, 5 products

Same inputs: stayed before → churned after.

C2 · 11 logins, 1 product

Same inputs: stayed before → churned after.

Invented, deliberately simplified classifier and mature cohorts. Real class boundaries overlap; no real bank performance is reported.

BANK EXAMPLE C1 has 5 logins and 5 products; C2 has 11 logins and 1 product. The same inputs now accompany different observed churn outcomes.

BLOCK 1 / MONITORING & DEGRADATION

Input checks now; quality checks later

The target window creates a delay between a prediction and complete outcome evidence.

LINK EVIDENCE TO THE ORIGINAL PREDICTION DATE AND TARGET WINDOW



AVAILABLE AT D0

Operational checks

Freshness + missingness

Coverage + completion

WAIT FOR EVIDENCE

Unknown outcomes

No full 30-day label

Absence ≠ stayed

MATURE LABELS

Quality review

Join scores to labels

Evaluate by cohort

Save customer, cutoff, target window, features, M1 version, and campaign action context.

The 30-day window and dates are teaching choices. Ingestion can delay evaluation further.

Key concepts

- **At scoring time:** check freshness, missingness, execution, and coverage.
- **During the window:** the full churn outcome is still unknown.
- **After ingestion:** join labels to the original score and target window.
- **Evaluate by cohort:** precision, recall, and calibration answer different questions.

An absent label does not mean the customer stayed.

BANK EXAMPLE

Ana’s D0 score is saved with its context. Complete next-30-day evaluation must wait for that window and ingestion.

BLOCK 1 / MONITORING & DEGRADATION

Drift detection: methods and their purpose

Methods compare inputs; they do not directly measure business value or model correctness.

Method	Useful interpretation	Key limitation
PSI	Compare proportions in fixed bins	Depends on binning and reference
KS	Compare continuous distributions	Sample size affects sensitivity
KL divergence	Directional distribution difference	Needs careful support handling
JS divergence	Symmetric distribution comparison	Still does not measure accuracy

Key concepts

- **Reference:** name the population, period, and feature definition.
- **Thresholds:** calibrate for the application rather than copying a universal rule.
- **Segments:** inspect affected cohorts instead of relying only on an aggregate.
- **Action:** investigate drift alongside quality, change, and business evidence.

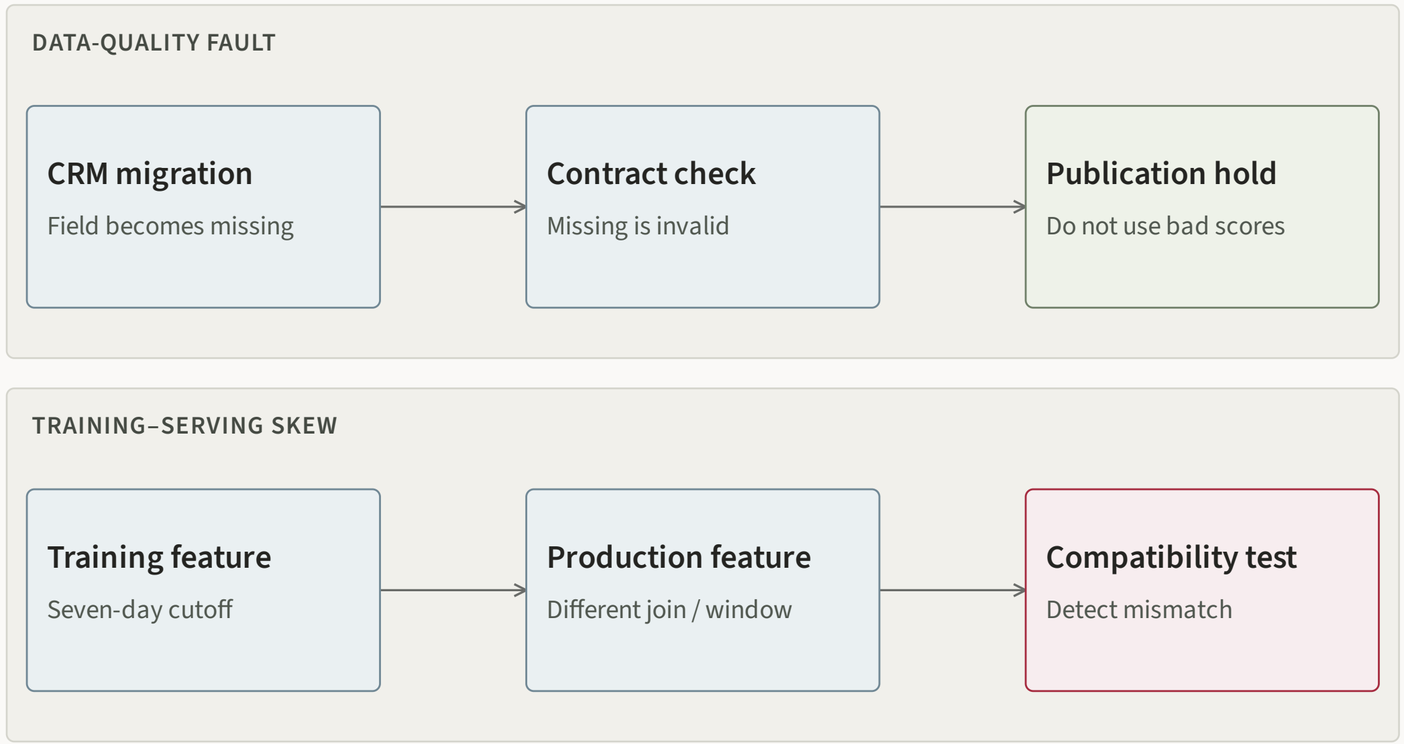
Interpret the signal alongside data validity, sample size, and downstream quality.

BANK EXAMPLE Compare the same seven-day login feature before and after a campaign; investigate why the cohort mix changed.

BLOCK 1 / MONITORING & DEGRADATION

Data-quality faults and training-serving skew

Invalid data and inconsistent preparation differ from a shifted valid population.



These are different diagnoses. Both can occur even when the model artifact is unchanged.

Key concepts

- **Data-quality fault:** missing, stale, corrupt, or incompatible records.
- **Skew:** training and production prepare the feature differently.
- **Inspect:** definitions, cutoffs, joins, source changes, and code versions.
- **Repair the cause:** retraining on defective inputs can preserve the defect.

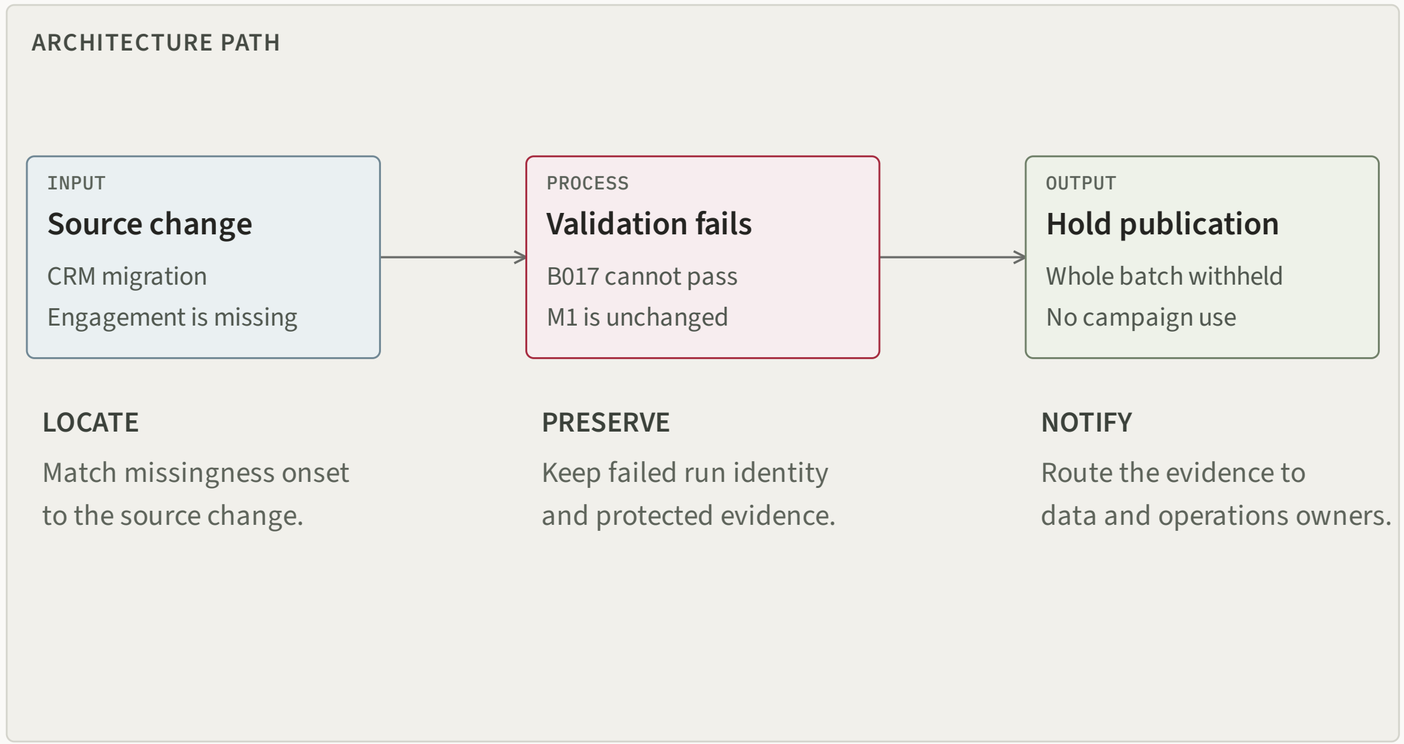
Do not use the word “drift” as a diagnosis for every alarm.

BANK EXAMPLE A missing engagement field after CRM migration is invalid input. A 30-day production count replacing seven days is skew.

BLOCK 1 / MONITORING & DEGRADATION

A publication gate contains a bad batch

The CRM incident illustrates a concrete input fault and a controlled response.



This is an invented incident. No changed score or customer outcome is claimed.

Key concepts

- **Detect:** invalid records violate the agreed input contract.
- **Contain:** hold the entire B017 batch before publication.
- **Investigate:** relate cohort, source change, feature definition, and execution.
- **Keep M1:** a source fault does not itself justify a new learned model.

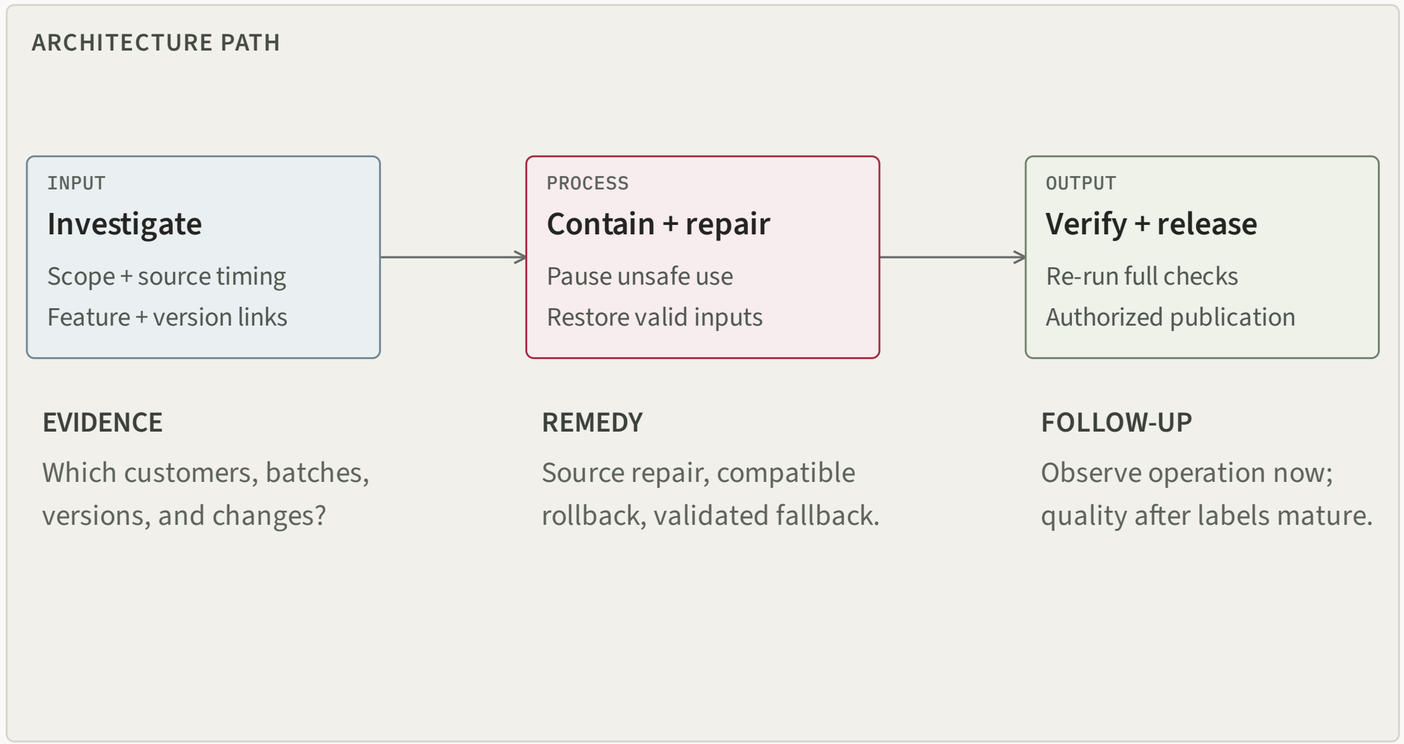
A gate blocks unsafe use; an alert sends evidence to someone who can investigate.

BANK EXAMPLE Ana’s engagement value is missing, not zero. No new score or completed customer contact is invented.

BLOCK 1 / MONITORING & DEGRADATION

Respond: investigate, contain, repair, verify

A monitoring signal opens a decision process; it does not select or approve a remedy.



Candidate training is optional and separately justified; promotion remains a separate release decision.

Key concepts

- **Diagnose** before choosing repair, rollback, fallback, or training.
- **Preserve evidence** and the identity of failed attempts.
- **Validate the complete path** after the chosen repair.
- **Request a model change** only when evidence supports one.

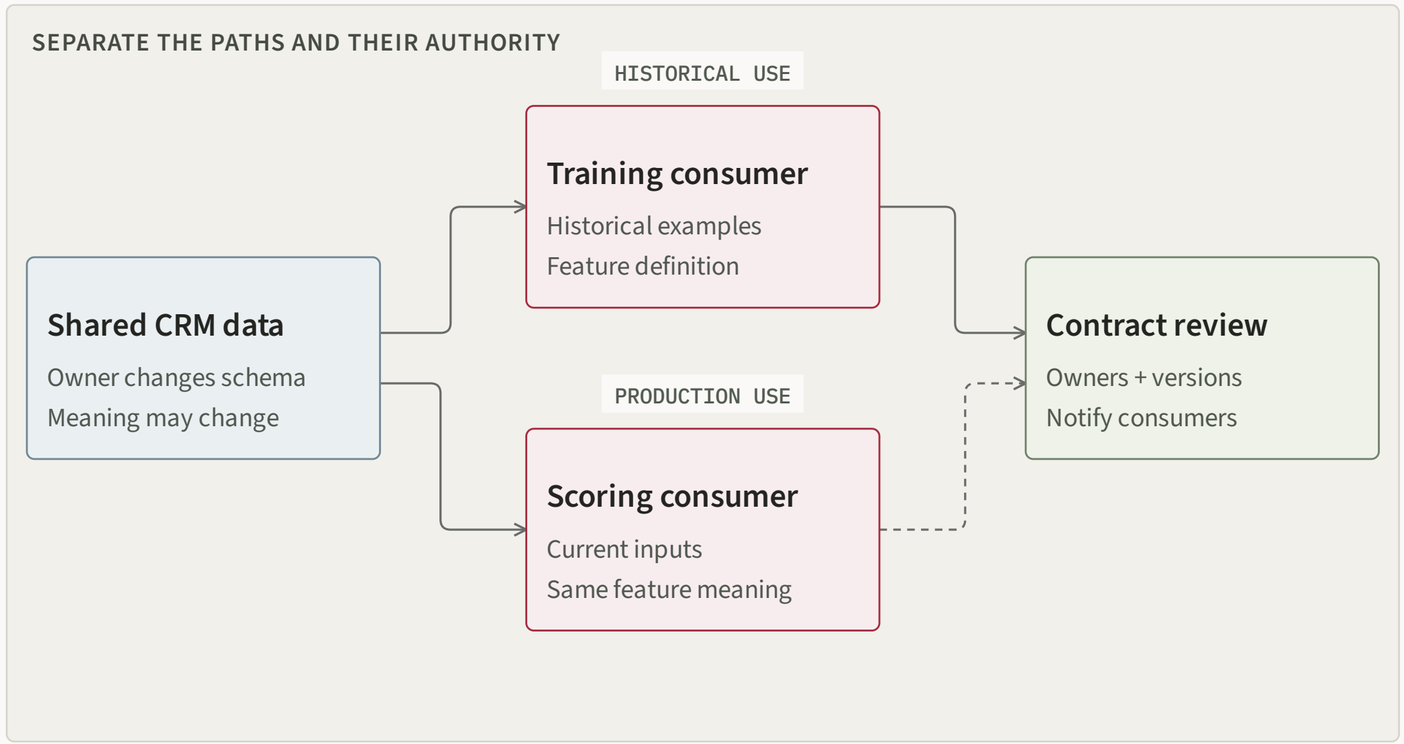
Handoff to Luca: which dependencies made this failure possible?

BANK EXAMPLE Repair the CRM path, validate a new execution B017-R1, and retain M1 unless a separate model change is justified.

BLOCK 1 / DESIGN DECISIONS

Hidden dependencies create technical debt

An innocent upstream change can alter ML behavior without changing the model.



A dependency needs an owner and change contract; documenting an arrow is not enough.

Key concepts

- **Hidden coupling:** features depend on sources owned by other teams.
- **Pipeline complexity:** parallel preparation paths can diverge.
- **Versioning gaps:** a model artifact alone cannot reproduce a decision.
- **Context matters:** architecture decisions depend on requirements and constraints.

Make data dependencies as reviewable as code dependencies.

BANK EXAMPLE — CRM changes its engagement export for another purpose; both training and production can inherit the change.

BLOCK 1 / DESIGN DECISIONS

Make the feature contract explicit

A contract records meaning and ownership as well as structural schema.

Contract field	Teaching definition
Meaning	Count of app logins in the seven days ending at the scoring cutoff
Structure	Non-negative count; declared type and units
Missingness	Missing is incompatible with this normal scoring path
Ownership	Source and feature owners review semantic changes
Compatibility	Train and score with tested, linked definitions

Key concepts

- **Semantics:** define what the feature actually measures.
- **Time:** specify window, cutoff, and joins consistently.
- **Validity:** define types, ranges, freshness, and missingness.
- **Ownership:** review upstream changes with affected consumers.

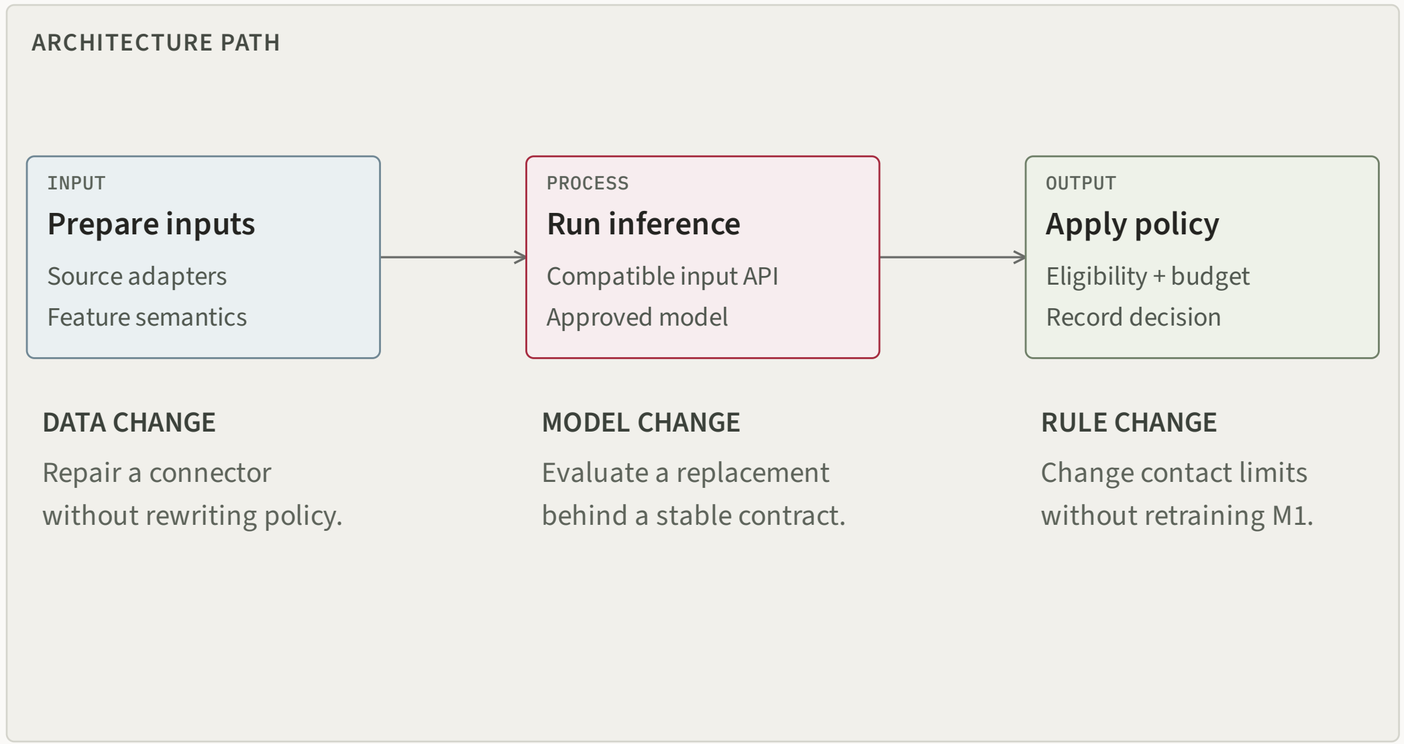
Schema checks alone cannot detect every change in meaning.

BANK EXAMPLE engagement_7d means app logins during seven days ending at the scoring cutoff; missing must not silently become zero.

BLOCK 1 / DESIGN DECISIONS

Separate responsibilities where change happens

Clear interfaces let different concerns evolve without unnecessary relearning.



Logical separation can remain inside one application. Microservices need a separate justification.

Key concepts

- **Single responsibility:** identify each module's reason to change.
- **Interfaces:** specify data and behavior at each boundary.
- **Dependency injection:** provide implementations through explicit contracts.
- **Deployment:** logical modules do not automatically require separate services.

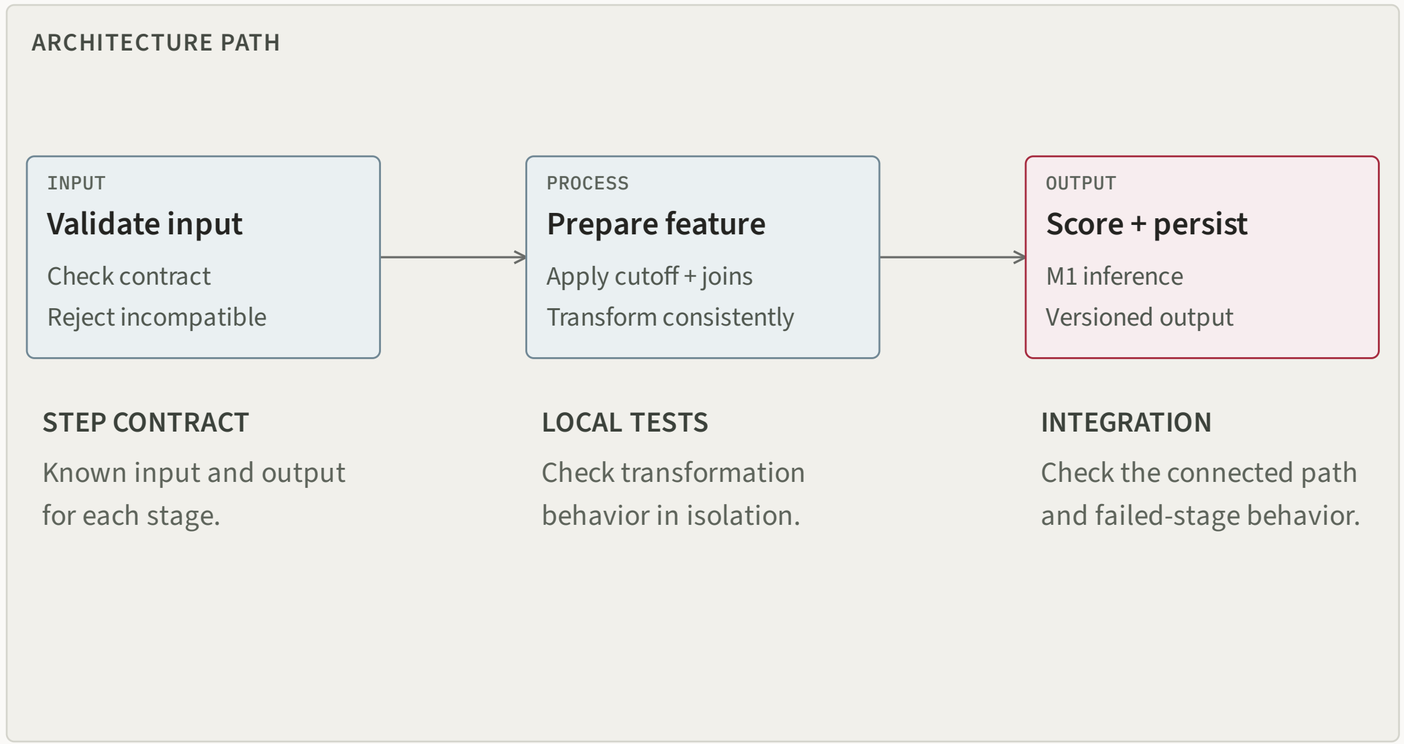
Separate concerns first; choose deployment units from the actual trade-offs.

BANK EXAMPLE A connector fix changes preparation; a contact-frequency rule changes campaign policy; a new M2 changes inference.

BLOCK 1 / DESIGN DECISIONS

Pipeline pattern: order the responsibilities

Make preparation and validation steps explicit and testable.



A pipeline structures work; its error and publication policies still require explicit design.

Key concepts

- **Explicit stages** clarify transformations and dependencies.
- **Stage contracts** make intermediate data understandable.
- **Tests** cover local behavior and end-to-end compatibility.
- **Failure handling** prevents a partial result from masquerading as a complete batch.

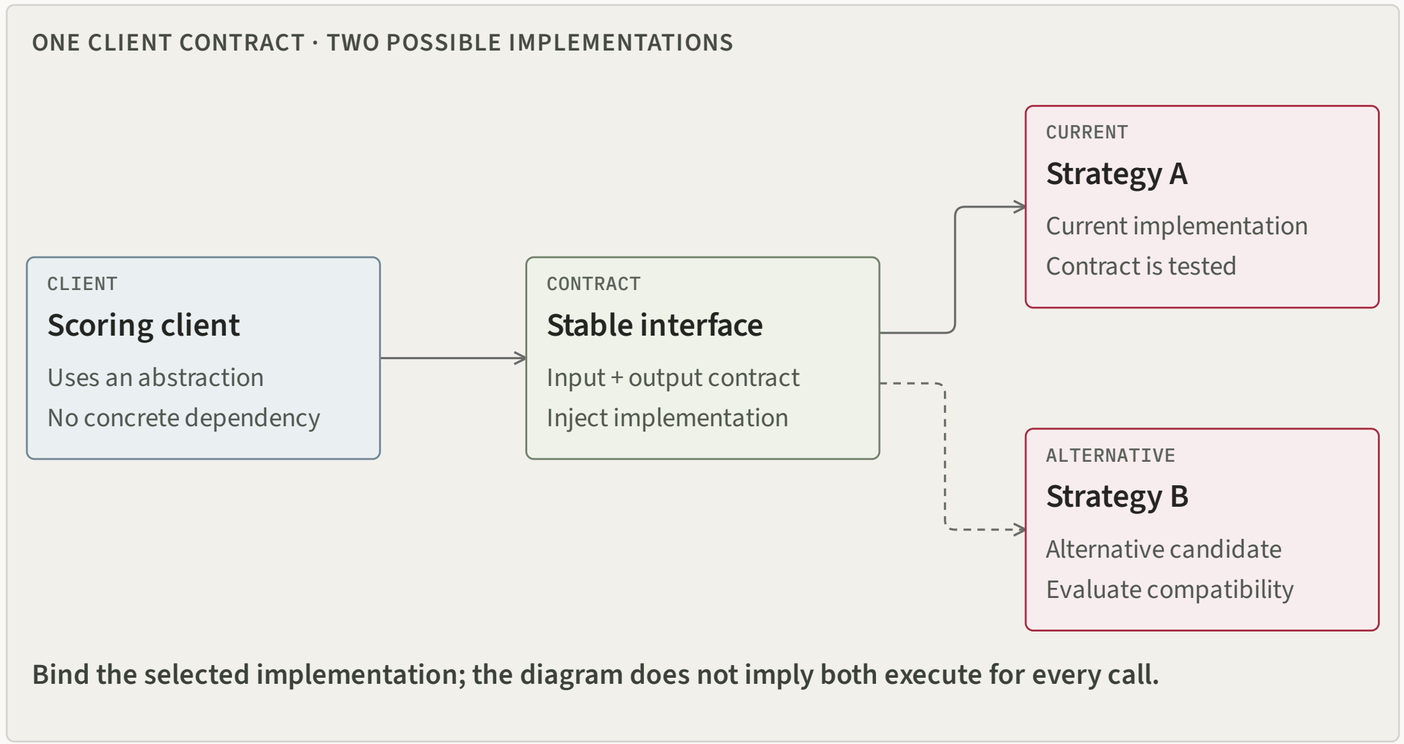
Use structure to expose the dependency, not to hide it in orchestration code.

BANK EXAMPLE Validate CRM records before computing engagement_7d and invoking M1.

BLOCK 1 / DESIGN DECISIONS

Strategy pattern: replace one implementation

A stable interface separates the client from a specific implementation choice.



Substitution in code still requires semantic compatibility, predictive evaluation, and release approval.

Key concepts

- **Stable interface:** the client relies on a contract, not a specific class.
- **Alternative implementations** can be supplied through that interface.
- **Compatibility** still includes meaning, not only types.
- **Evaluation** and release approval still apply when behavior changes.

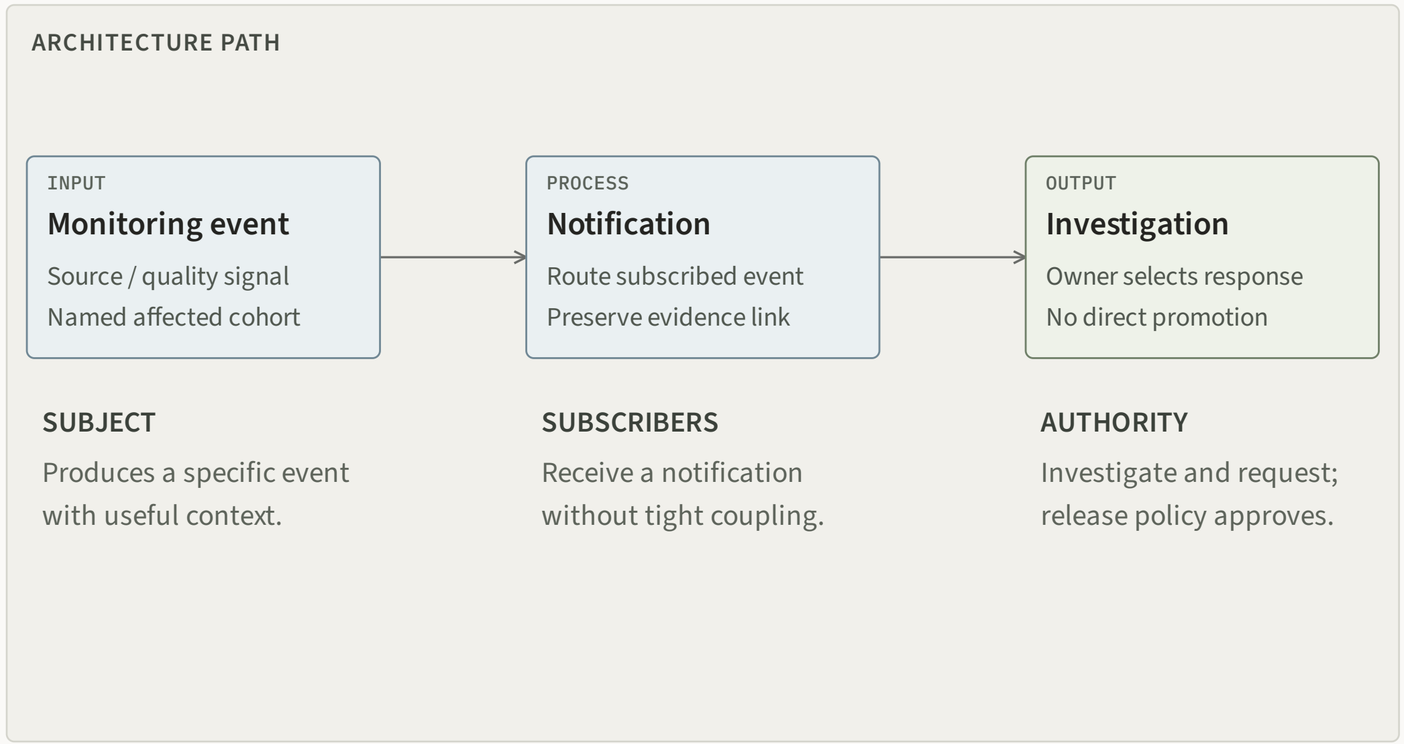
Interchangeability in code is not equivalence in predictive behavior.

BANK EXAMPLE Replace a feature-preparation implementation behind a stable interface; verify equivalent semantics before scoring.

BLOCK 1 / DESIGN DECISIONS

Observer pattern: notify, then investigate

A change notification decouples the signal producer from interested responders.



Do not wire a generic drift alarm directly to unreviewed model promotion.

Key concepts

- **Subject:** emits a meaningful change event.
- **Subscribers:** receive notifications without tightly coupling the producer.
- **Context:** include cohort, reference, and evidence links.
- **Authority:** receiving a notification does not grant release privileges.

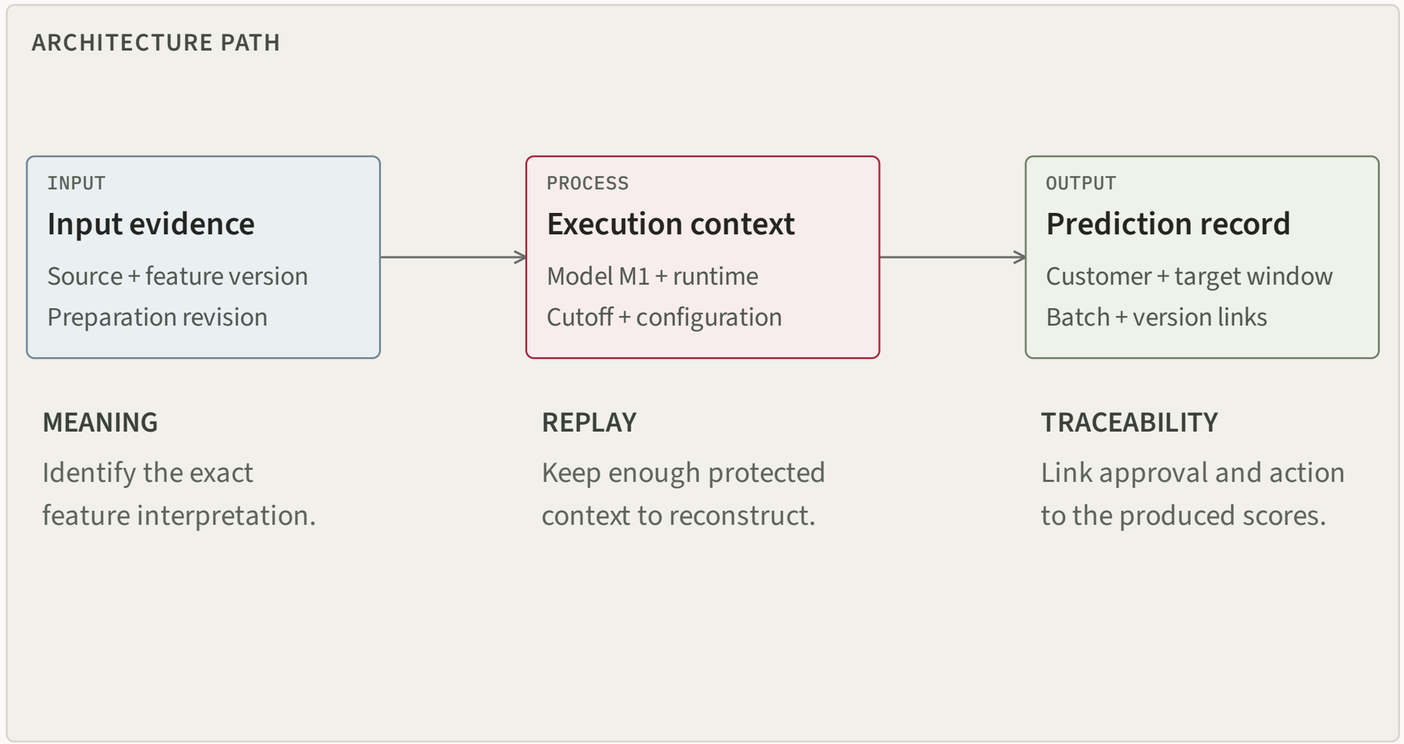
A useful notification requests attention and preserves the decision boundary.

BANK EXAMPLE A missingness event notifies data and operations owners; it does not instruct the registry to promote M2.

BLOCK 1 / DESIGN DECISIONS

Version the system, not only the model

A prediction depends on a compatible set of data meaning, code, model, and configuration.



Protect sensitive evidence; a traceability link need not expose raw customer history to everyone.

Key concepts

- **Data meaning:** record source and feature definitions.
- **Code:** identify preparation and scoring revisions.
- **Model:** identify artifact, runtime, and configuration.
- **Outputs:** connect the compatible set to score and decision records.

Co-versioning connects system and model lifecycles.

BANK EXAMPLE M1 alone cannot explain Ana’s score if a CRM migration changed the meaning of its engagement input.

BLOCK 1 / DESIGN DECISIONS

Modules or services? Make the trade-off

Independent deployment is useful only when it justifies its operating cost.

Concern	Modules in one application	Separate services
Release	Coordinated release	Independent release where justified
Failure	Fewer remote dependencies	Network, protocol and partial failures
Operation	Simpler local observability	Distributed tracing and recovery
Scaling	Scale shared runtime	Scale selected capabilities
Choice	Useful simple starting point	Autonomy must justify complexity

Key concepts

- **Modules** can provide strong logical boundaries with simpler operation.
- **Services** can support independent change and scaling.
- **Distribution** adds failure modes and version coordination.
- **Evidence** and requirements should drive the choice.

There is no universal best ML architecture.

BANK EXAMPLE Separate preparation from policy logically; deploy them independently only when the bank's needs justify it.

BLOCK 2 / ARCHITECTURAL TACTICS

Architectural tactics connect concerns to controls

Use explicit mechanisms to address uncertainty, change, recovery, and auditability.

Concern	Architectural tactic	Bank effect
Data change	Contracts + validation boundary	Detect incompatible CRM inputs
Predictive uncertainty	Evaluation + reviewed release	Use evidence for model changes
Recovery	Compatible rollback / fallback	Restore the usable path
Auditability	Co-versioning + protected lineage	Reconstruct a score and action
Privacy / security	Scoped access + minimization	Limit exposure and authority

Key concepts

- **Separation** isolates data preparation, model execution, and business policy.
- **Observation** makes operational and predictive behavior reviewable.
- **Recovery** restores a compatible path rather than only one artifact.
- **Governance** links evidence, approval, access, and responsibility.

A tactic is credible when its mechanism, owner, and evidence are clear.

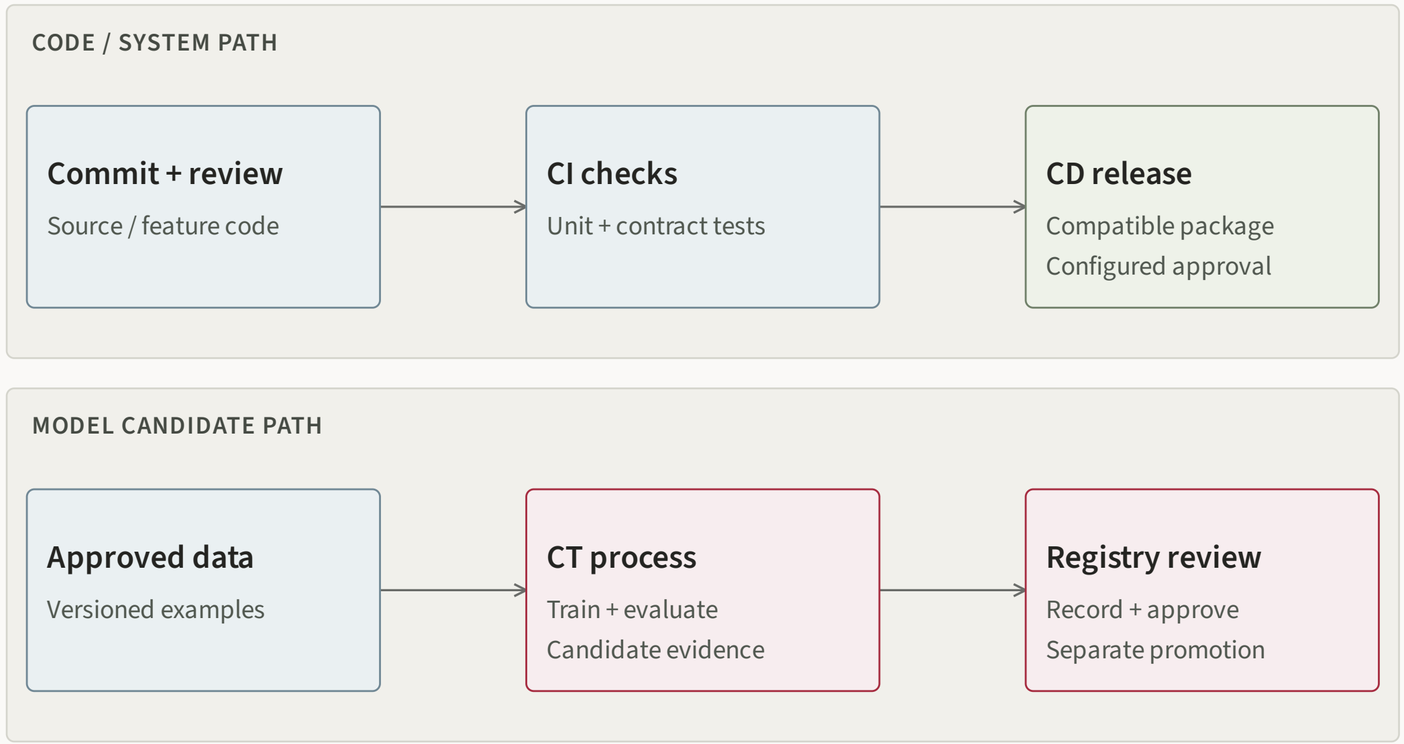
BANK EXAMPLE

A validation boundary contains a bad CRM batch; version links and approvals support controlled repair.

BLOCK 2 / DEVOPS & MLOPS

CI, CD, and CT need different evidence

Integration, delivery, deployment, and training automate different responsibilities.



Continuous delivery keeps a change releasable; continuous deployment automatically releases gated changes.

Key concepts

- **CI:** integrate changes and run code, contract, and path checks.
- **Delivery:** keep a compatible change ready for an approval decision.
- **Deployment:** automatically release changes passing the defined policy.
- **CT:** train and evaluate candidates; completion is not production approval.

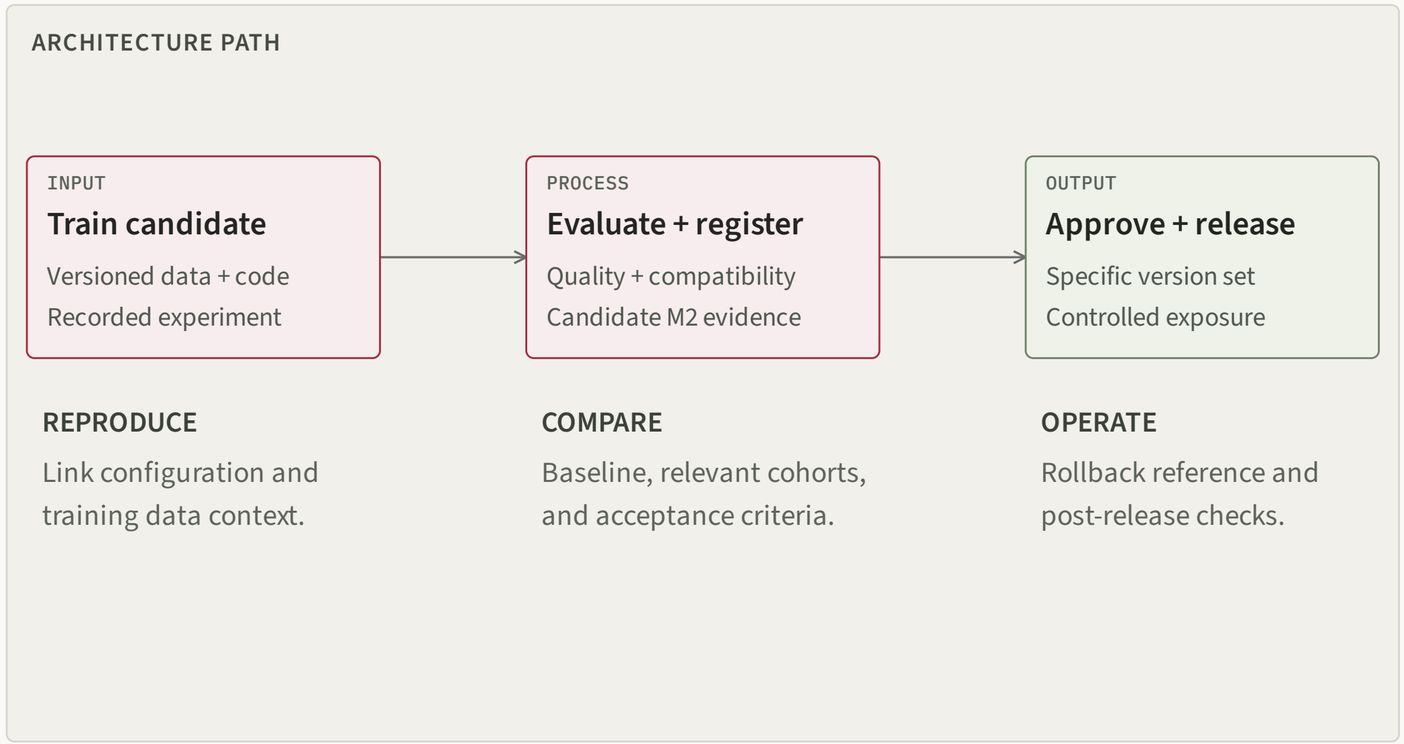
Automation executes policy; it does not make acceptance criteria identical.

BANK EXAMPLE A connector repair can use CI/CD while retaining M1. Retraining is not required for every pipeline change.

BLOCK 2 / DEVOPS & MLOPS

Candidate training ends before promotion

A registry stores candidate evidence; a separate release decision controls production use.



A drift signal may request review or training. It does not authorize release.

Key concepts

- **Reproducibility:** record data, code, configuration, and artifacts.
- **Evaluation:** compare candidate evidence against the baseline and relevant cohorts.
- **Registration:** preserve identity and lineage for review.
- **Promotion:** authorize a specific compatible set with recovery planned.

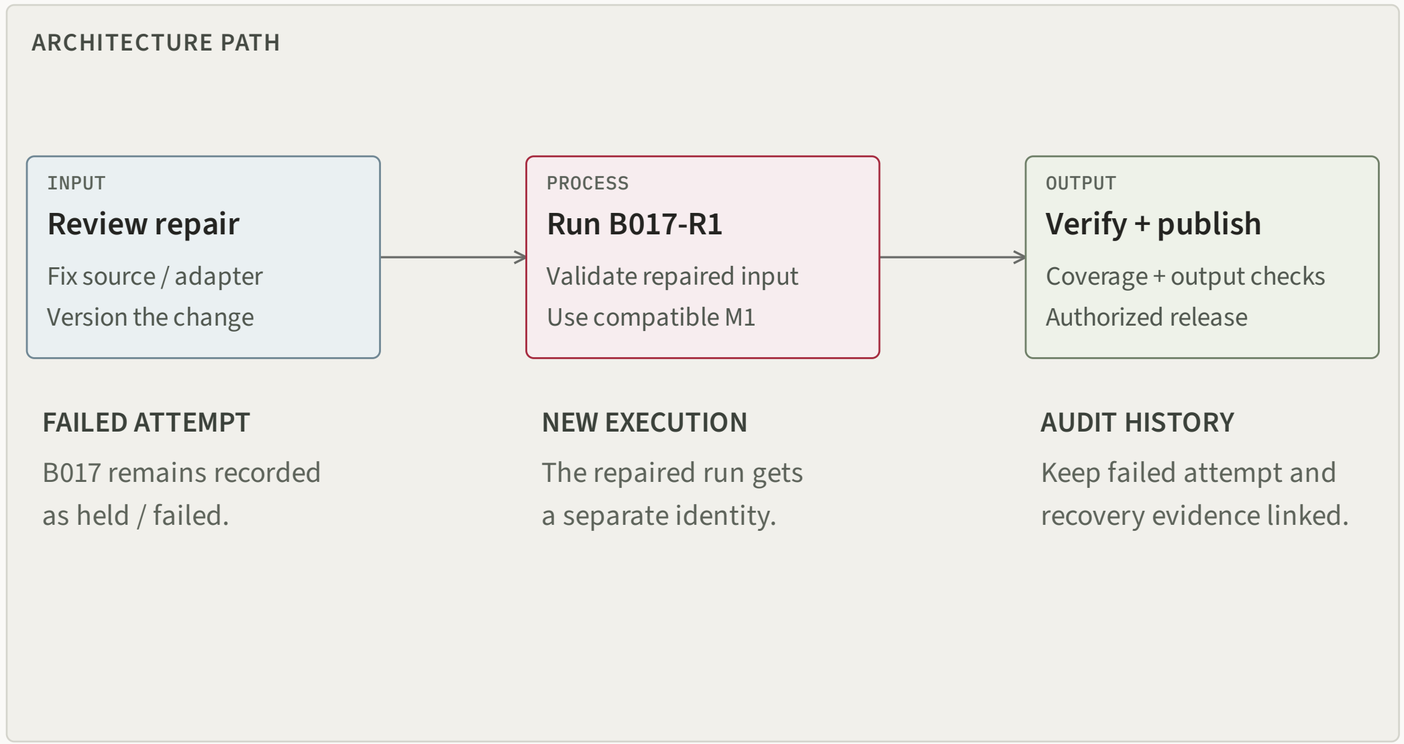
The registry is part of governance, not the whole approval process.

BANK EXAMPLE M2 remains a candidate until reviewed. M1 remains the approved scoring artifact during the CRM repair.

BLOCK 2 / DEVOPS & MLOPS

Repair the input path; retain M1 when appropriate

The CRM incident requires a verified source repair, not an assumed new model.



Do not overwrite failed evidence. Retain M1 unless separate evidence justifies changing it.

Key concepts

- **Repair** the source or preparation defect.
- **Version** the reviewed fix and compatible configuration.
- **Re-run** under a new execution identity and validate the complete path.
- **Publish** only after the configured evidence and approval requirements pass.

A successful fix needs evidence at the consumer boundary, not only a passing connector test.

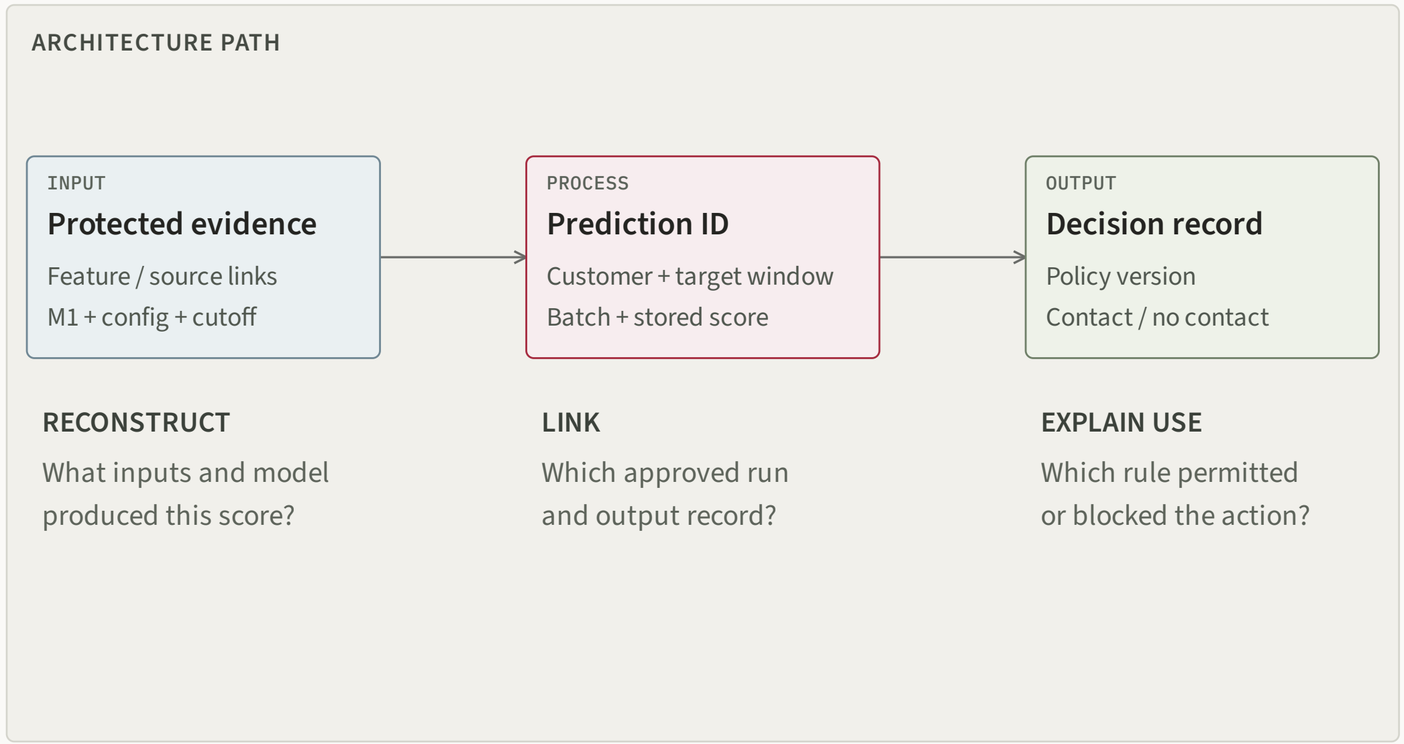
BANK EXAMPLE B017 is the held attempt. B017-R1 links the reviewed fix, compatible M1, validation evidence, and publication approval.

Bass, §14.1 · Kalinowski, §16.3–16.5 (supporting) · Simon's notes, §4

BLOCK 2 / DEVOPS & MLOPS

Trace the prediction and the decision

Lineage connects input evidence, execution, approval, and downstream action.



Protected references can support audit without exposing all customer history in campaign tools.

Key concepts

- **Prediction identity** links customer, time, and target window.
- **Execution evidence** identifies compatible code, features, model, and config.
- **Decision evidence** records policy version and action.
- **Access controls** protect sensitive reconstruction material.

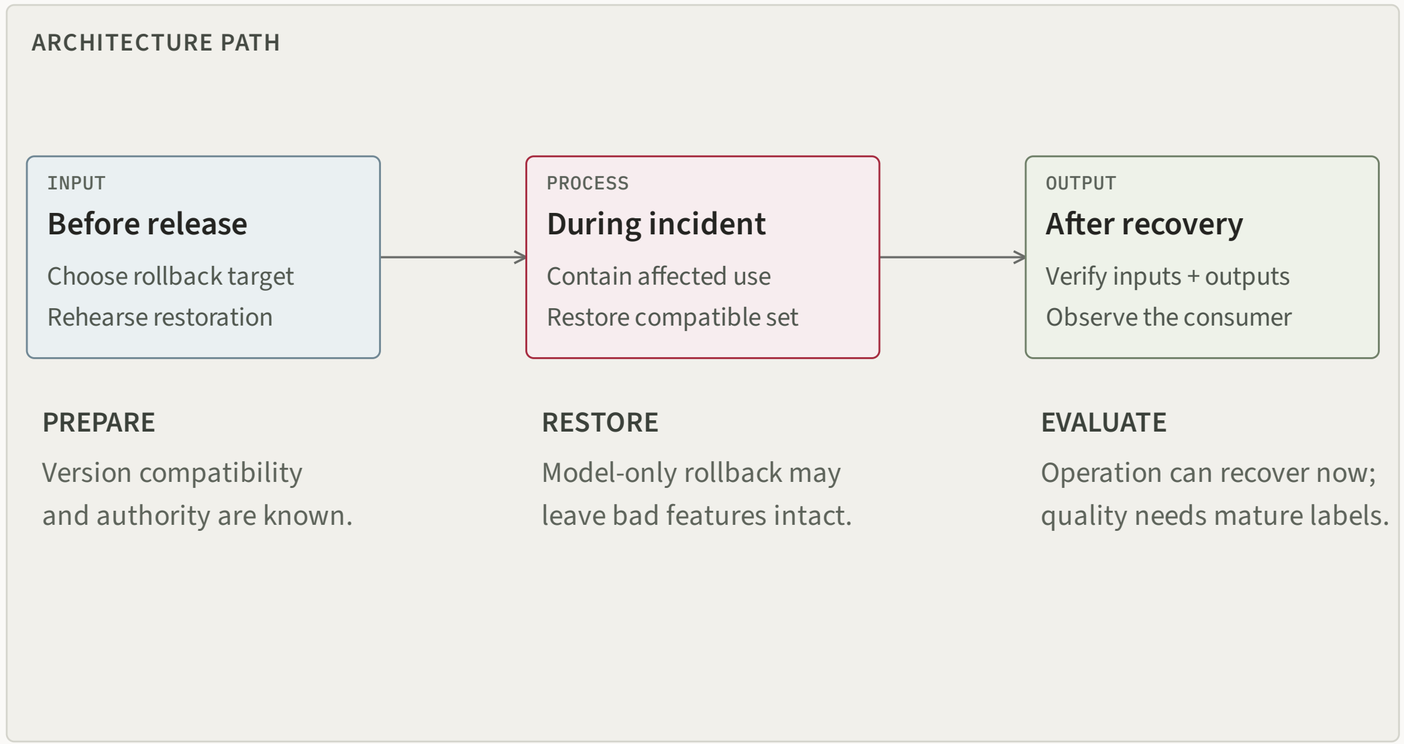
A model version alone cannot reconstruct the full decision.

BANK EXAMPLE An investigator can trace Ana’s original score and the campaign’s separate no-contact or contact decision.

BLOCK 2 / DEVOPS & MLOPS

Recovery restores a compatible path

Rehearse restoration before release, and verify it at the consumer boundary.



Historical customer contacts cannot be undone. Recovery of the system does not erase prior effects.

Key concepts

- **Know the target:** identify the last compatible release or validated fallback.
- **Rehearse:** test the recovery procedure before relying on it.
- **Restore the set:** include data meaning, runtime, and configuration.
- **Verify:** inspect stored scores and actual consumer behavior.

Handoff to security: who may read, change, approve, or operate these paths?

BANK EXAMPLE — Rolling M1 back cannot repair a CRM field that is still missing; the input path must be restored.

BLOCK 2 / SECURITY & PRIVACY

Security threats depend on attacker access

Locate the attacker’s control before naming the ML attack mechanism.

Access surface	Mechanism	What changes
Training records / learning inputs	Poisoning	The learned behavior
Production prediction inputs	Inference-time manipulation	The input to a fixed model
Outputs or model access	Privacy inference	Attacker knowledge of sensitive data

Key concepts

- **Poisoning:** altered learning inputs influence the trained model.
- **Inference manipulation:** altered production inputs affect an existing model’s output.
- **Privacy inference:** access can reveal sensitive membership or attributes.
- **Assumptions matter:** access, knowledge, and goals differ by threat.

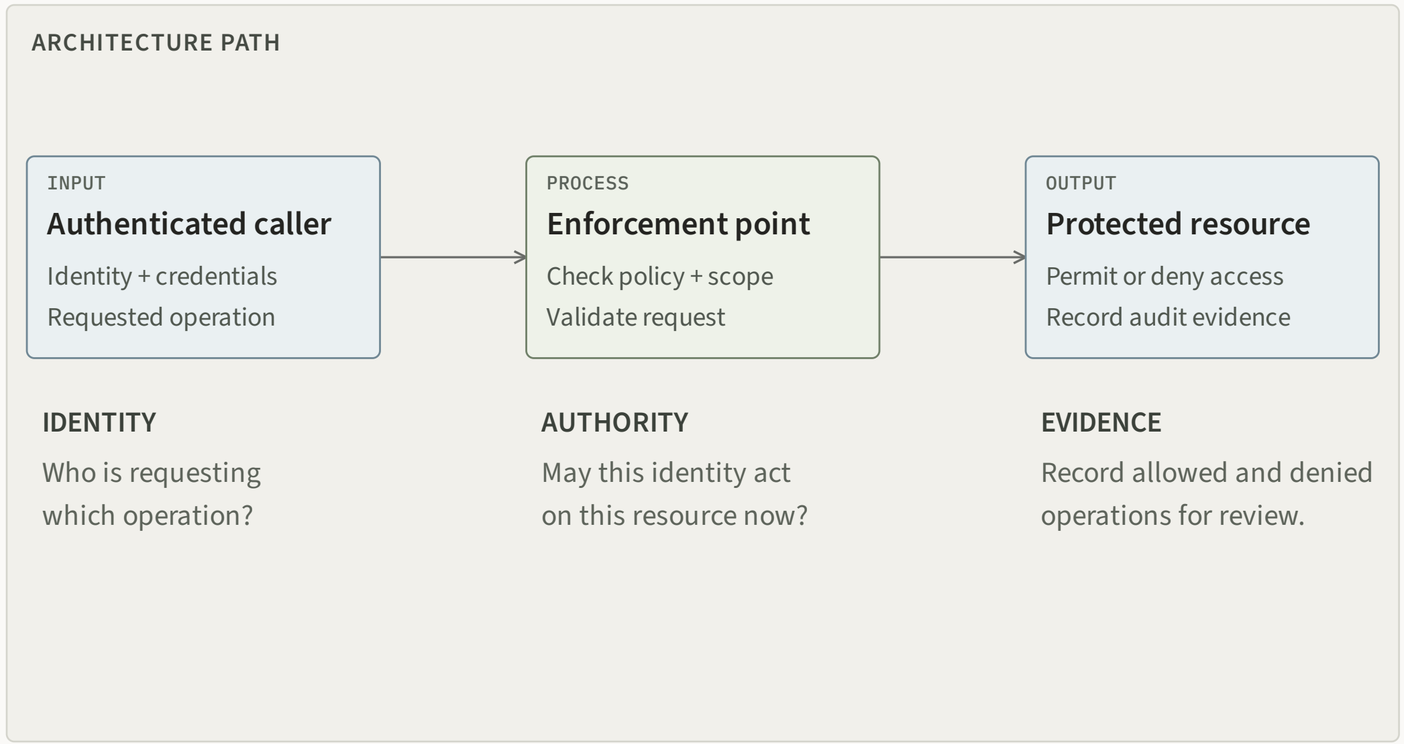
The accidental CRM migration remains a different scenario.

BANK EXAMPLE A compromised CRM writer could affect current inputs; those records create a poisoning path only if later used for training.

BLOCK 2 / SECURITY & PRIVACY

Enforce access at the resource boundary

Authentication, authorization, validation, and audit solve different parts of the problem.



Logical enforcement functions need not be separate microservices. Network reachability is not authorization.

Key concepts

- **Authenticate:** establish caller identity.
- **Authorize:** check operation, resource, and scope.
- **Validate:** enforce the input contract independently.
- **Audit:** preserve useful access and change evidence.

A trust boundary needs an enforced policy, not only a box on a diagram.

BANK EXAMPLE The scorer may read the approved M1 artifact; it must not gain permission to promote M2 merely by reaching the registry.

BLOCK 2 / SECURITY & PRIVACY

Least privilege follows responsibilities

Separate source writes, candidate creation, release approval, and campaign use.

Identity	Read	Write / decide	Excluded authority
Feature job	Approved source	Prepared features	Promote a model
Trainer	Training features	Candidate artifact	Release to production
Release reviewer	Candidate + checks	Approve a version set	Rewrite training data
Batch scorer	Approved M1 + inputs	Versioned scores	Change campaign policy
Campaign user	Approved score view	Eligible contact action	Read unrestricted history

Key concepts

- **Scope identities** to the operations and resources they need.
- **Separate duties:** training does not imply release authority.
- **Protect credentials** and avoid secrets in code or broad logs.
- **Review changes:** connect access, source updates, and approvals.

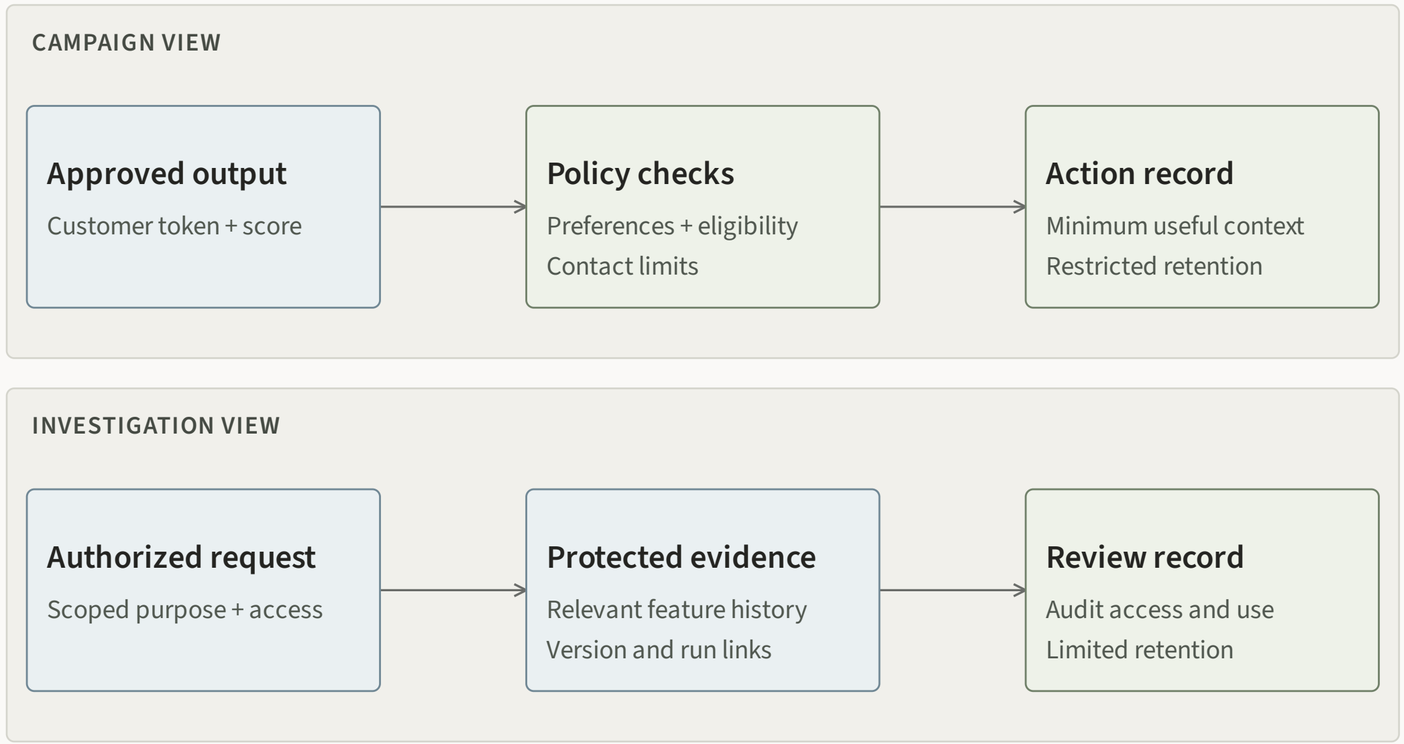
This abbreviated teaching matrix is a starting point, not a complete bank permission design.

BANK EXAMPLE A campaign user reads an approved score view, not unrestricted training data or production deployment credentials.

BLOCK 2 / SECURITY & PRIVACY

Privacy: expose what each role needs

Campaign use and incident investigation require different views of customer information.



A token alone does not make real customer information anonymous.

Key concepts

- **Minimize collection** and the information exposed in outputs.
- **Separate views:** campaigns and investigators have different purposes.
- **Protect logs** and define retention and access rules.
- **Preserve accountability** without making raw evidence broadly available.

Privacy is an access-and-data-use design, not a label attached to a technique.

BANK EXAMPLE Campaign staff may need Ana’s approved score and eligibility context; investigators may need restricted versioned evidence.

BLOCK 2 / SECURITY & PRIVACY

Privacy techniques have specific limits

Choose a mechanism for a defined objective rather than claiming privacy by technique name.

Technique	Useful role	Limit to explain
Tokenization	Separate identifiers through a protected mapping	Does not establish anonymity
Synthetic data	Reduce direct use of original records	Can retain or leak sensitive patterns
Federated learning	Train without centralizing raw records	Updates and outputs can still leak
Differential privacy	Bound information about protected contributions	Needs unit, bounds and cumulative-loss accounting

Key concepts

- **Specify the objective:** what information should be protected from whom?
- **Tokens** and synthetic data do not automatically establish anonymity.
- **Federation** changes data location, not every leakage risk.
- **DP** needs a defined protected unit and accounting across releases; no derivation here.

Do not announce a guarantee without its assumptions and evidence.

BANK EXAMPLE Protecting one transaction differs from protecting all contributions of a bank customer.

BLOCK 2 / QUALITY CONCERNS

Use the quality checklist as a review tool

Connect concerns to architectural controls, ownership, and observable evidence.

Selected concern	Architecture review	Evidence example
SMN3 · Input monitoring	Compare appropriate input distributions	Cohort report + investigated change
SA1 · Recovery	Restore a compatible path	Recovery rehearsal + verified outputs
SS5 · Cyberattack defense	Enforce authorized operations	Rejected prohibited access / change
SS6 · Safety / fairness	Define harm and fairness targets	Group-specific review evidence
SP1 · Privacy consideration	Assess data use and privacy mechanisms	Access / retention / technique review

Key concepts

- **System and process concerns** both matter in ML-intensive systems.
- **Select relevant checks** rather than treating a short list as the entire checklist.
- **Map each concern** to a control, owner, and evidence.
- **Keep limits visible:** the separate corrigendum remains unread.

A checklist supports structured review; it is not a certification claim.

BANK EXAMPLE

Completeness validation addresses input quality; it is not by itself evidence of SS5 cyberattack defense.

BLOCK 2 / QUALITY CONCERNS

Predictive quality and system performance

Choose an acceptable model and system from several quality objectives.

Objective	Evidence	Architecture consequence
Predictive quality	Mature-label precision / recall / calibration	Candidate and cohort evaluation
Timeliness	Complete scores before campaign deadline	Batch capacity and dependencies
Cost	Training, inference and operations effort	Model complexity and infrastructure
Maintainability	Reproducibility, tests, recovery work	Versioned modular design

Key concepts

- **Accuracy** alone is an incomplete objective, especially with imbalanced outcomes.
- **Performance** includes the end-to-end deadline and resource use.
- **Complexity** must justify training and operating costs.
- **Acceptance** depends on the task and the consequences of errors.

Define acceptance criteria before selecting a more complex model.

BANK EXAMPLE A heavier churn model is useful only if its quality benefit justifies cost and still meets the campaign’s deadline.

Bucaioni, published Table 1, pp. 6–7 · Bass, Ch. 14 · Kalinowski, §13.4

BLOCK 2 / QUALITY CONCERNS

Robustness: invalid, unfamiliar, or ordinary?

Different inputs need different responses; do not treat every unusual value as the same failure.

Input condition	Evidence to inspect	Possible response
Invalid / incompatible	Contract, freshness, missingness	Reject / hold and investigate
Valid but unfamiliar	Population shift, coverage, OOD evidence	Review applicability; validated fallback
Valid and familiar	Normal input and execution checks	Score and monitor normally

Key concepts

- **Contract validity** and model applicability are distinct.
- **OOD inputs** may be structurally valid but poorly represented in training.
- **Confidence** alone is not a guaranteed detector of unfamiliar inputs.
- **Fallbacks** must be validated for the intended situation.

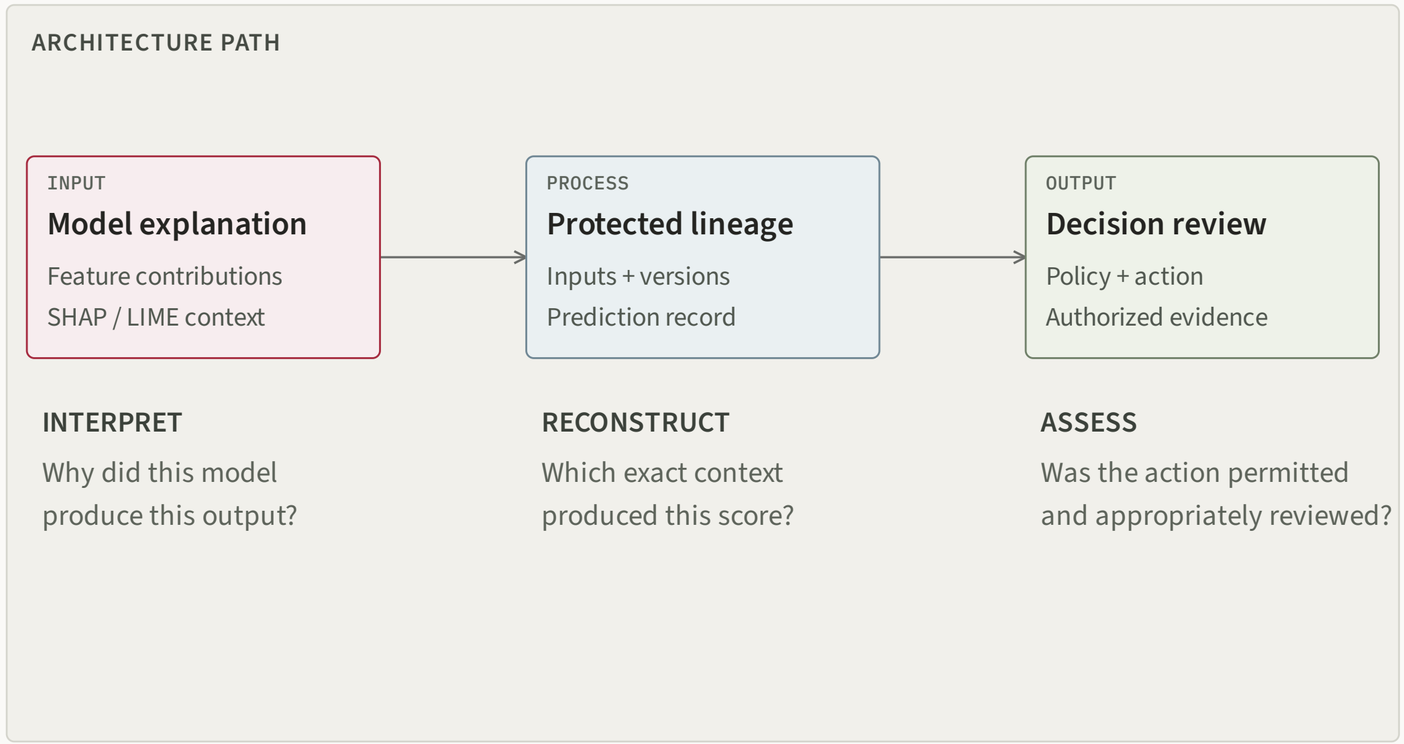
An architecture needs an explicit response when the model should not be trusted.

BANK EXAMPLE A missing activity value is invalid; a new customer population may be valid but outside the model's reliable coverage.

BLOCK 2 / QUALITY CONCERNS

Explainability and auditability differ

Explaining feature influence and reconstructing a historical decision are complementary tasks.



Feature attribution does not establish causality, fairness, or a customer's intention.

Key concepts

- **Explanation** helps interpret model behavior in context.
- **Auditability** needs historical evidence and traceable versions.
- **Decision review** includes business policy and action, not only the model.
- **Limits:** SHAP and LIME do not prove causality or fairness.

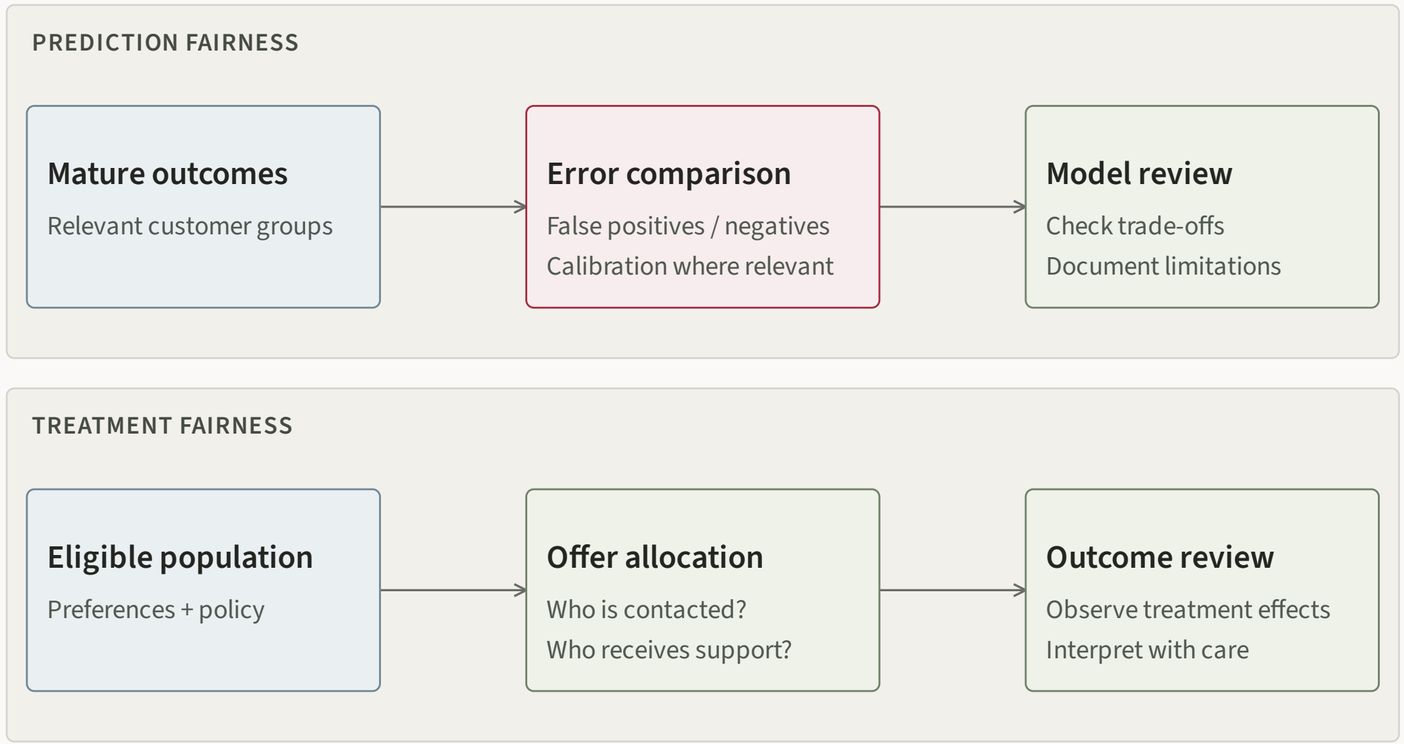
Protect evidence and test whether authorized reviewers can actually retrieve it.

BANK EXAMPLE An engagement contribution may help explain Ana's score; it cannot prove why she would leave the bank.

BLOCK 2 / QUALITY CONCERNS

Fairness: prediction versus treatment

Name what is being compared before selecting a metric or declaring an improvement.



Different fairness measures can conflict; define the objective and relevant groups in context.

Key concepts

- **Prediction fairness:** compare errors or calibration across relevant groups.
- **Treatment fairness:** inspect access to offers and policy effects.
- **Context:** select targets based on harms and requirements.
- **Trade-offs:** one metric does not settle every fairness concern.

Keep predictive quality and the consequences of intervention distinct.

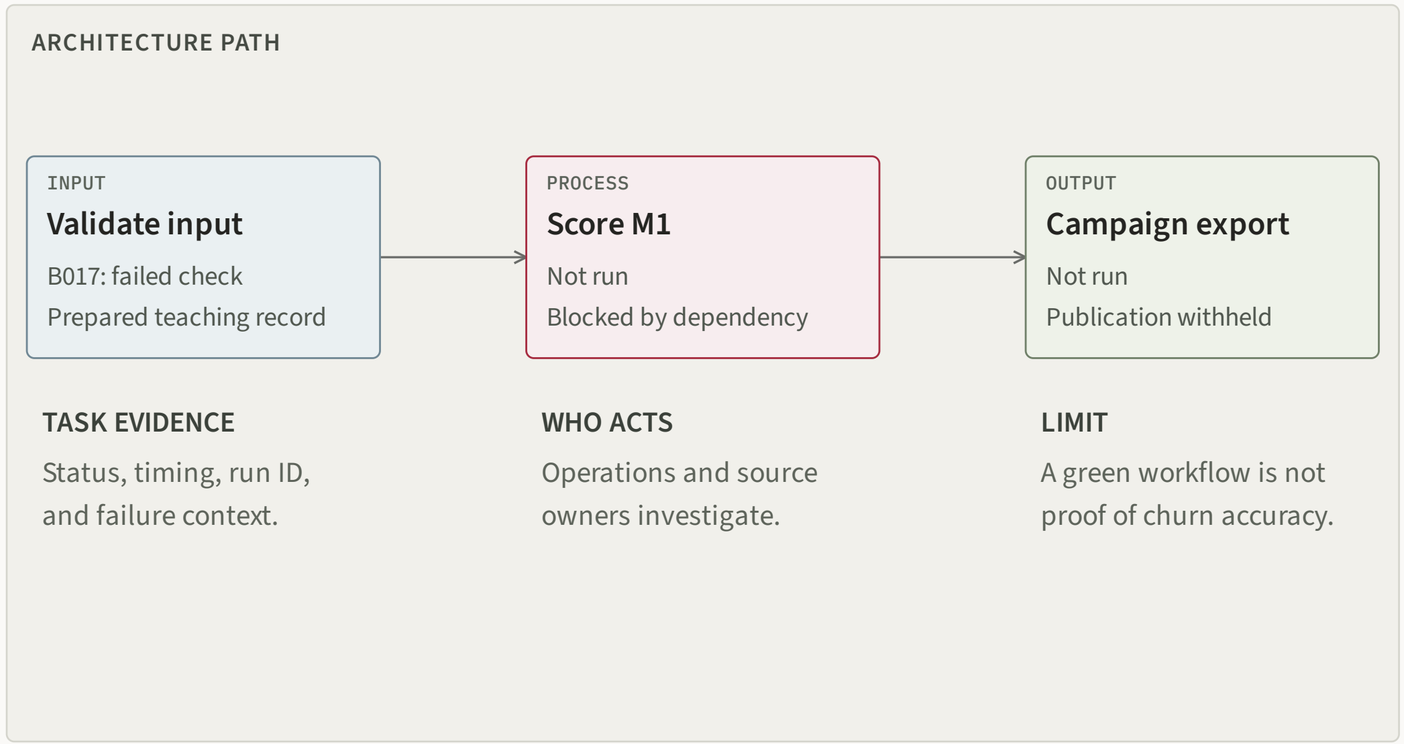
BANK EXAMPLE

Equal churn prediction errors do not imply equal access to retention offers under campaign allocation rules.

BLOCK 2 / PRACTICAL TOOLS

Airflow: make workflow dependencies visible

Orchestration supports task order, failure handling, and execution evidence.



Prepared teaching record, not a screenshot or a captured Airflow run. States are deliberately simplified.

Key concepts

- **Architecture role:** order dependent pipeline work.
- **Evidence:** task state, timing, and run context.
- **Action:** operators and source owners inspect failed dependencies.
- **Limit:** orchestration does not supply predictive or release approval evidence by itself.

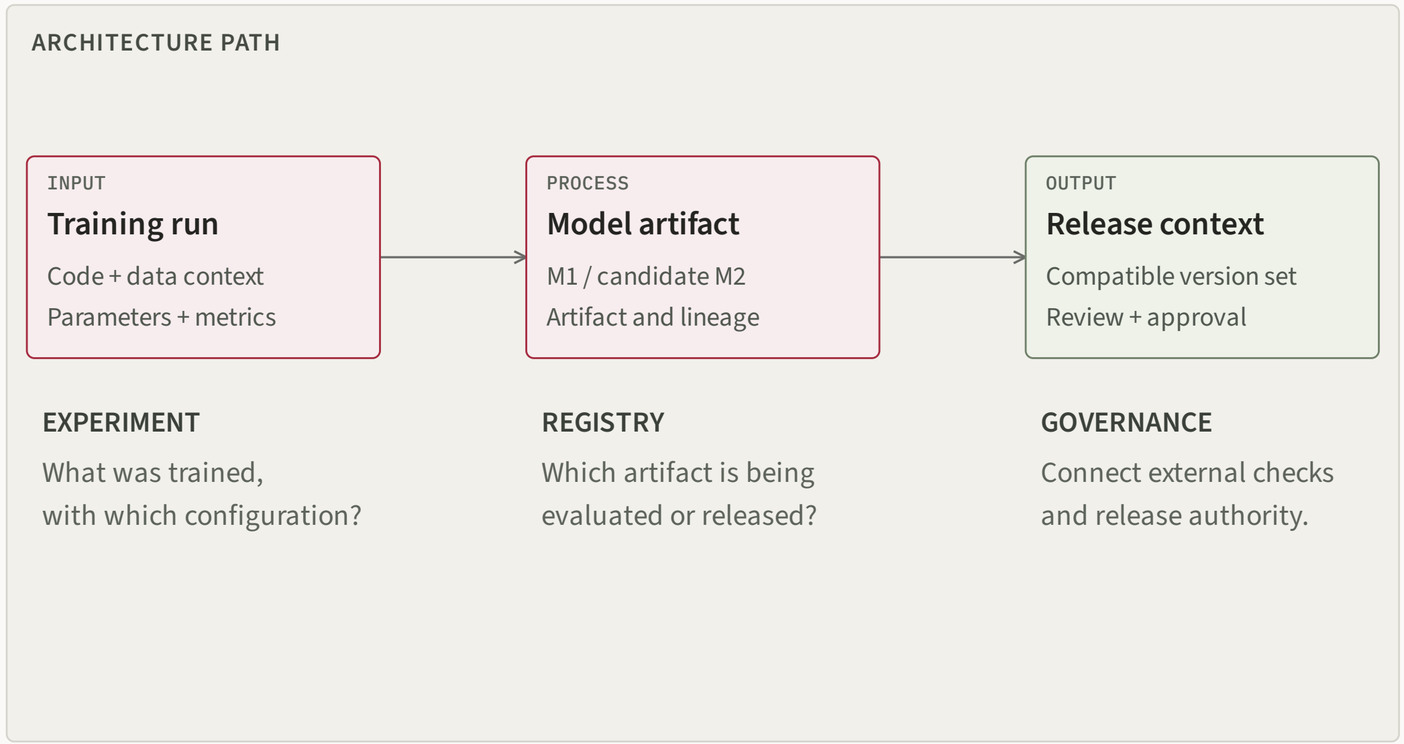
Connect each tool to a responsibility, evidence, and a response owner.

BANK EXAMPLE The required CRM validation must sit upstream of scoring and export so its failure blocks dependent work.

BLOCK 2 / PRACTICAL TOOLS

MLflow: connect experiments and artifacts

Model tracking supports reproducibility and review when evidence links are designed explicitly.



Illustrative records only. No experiment, registry transition, or production release was executed here.

Key concepts

- **Architecture role:** connect training evidence and model artifacts.
- **Evidence:** runs, configuration, metrics, and artifact identity.
- **Action:** reviewers compare candidates and verify compatible releases.
- **Limit:** no single tracking tool provides every governance control.

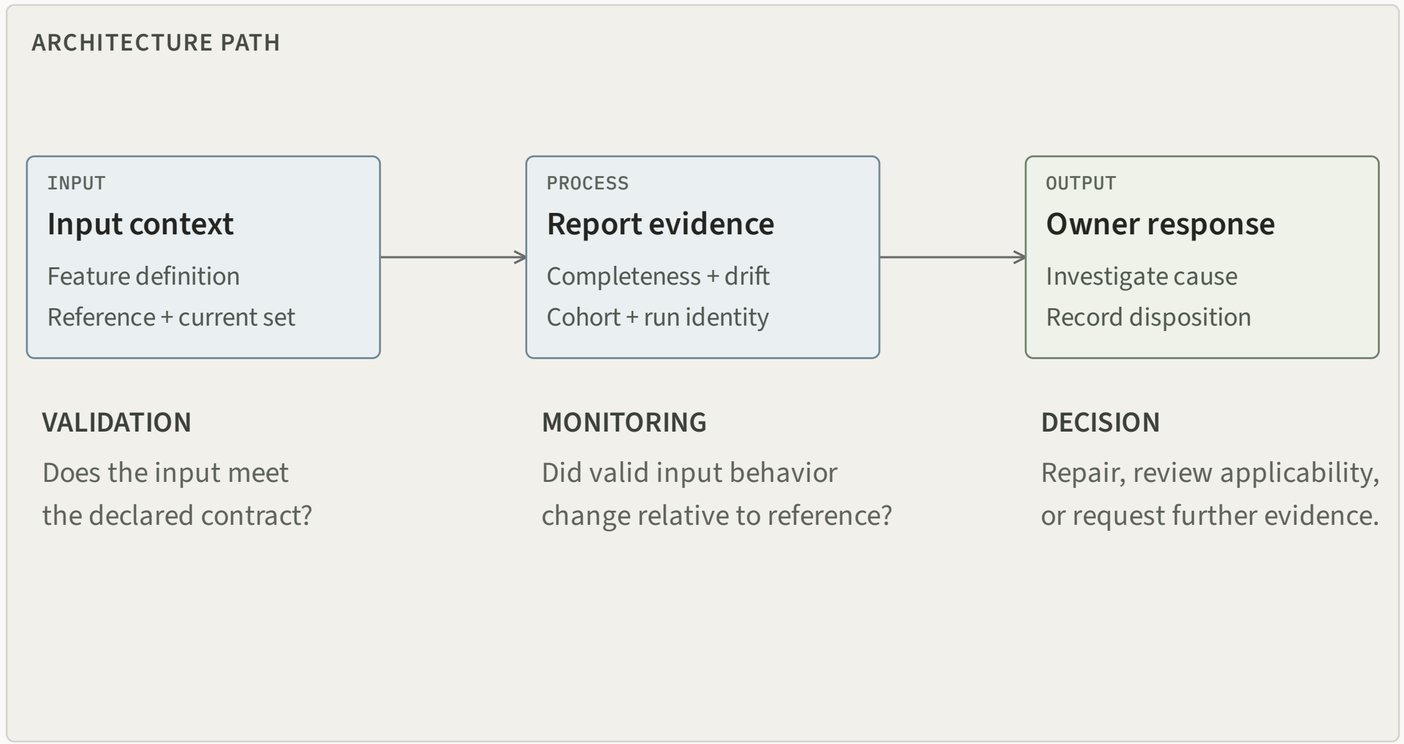
Tool records support the release decision; they do not replace its policy.

BANK EXAMPLE — Link the approved M1 to its evidence while recording B017-R1's separate input and execution context.

BLOCK 2 / PRACTICAL TOOLS

Validation and drift reports support diagnosis

A report is useful when its reference, feature meaning, cohort, and response are clear.



Reports here are teaching illustrations, not executed tool output.

Key concepts

- **Validation:** test declared expectations before use.
- **Drift comparison:** inspect change against a named reference.
- **Tool choice:** reports can support these roles; keep policy outside the screenshot.
- **Action:** preserve evidence and assign a response owner.

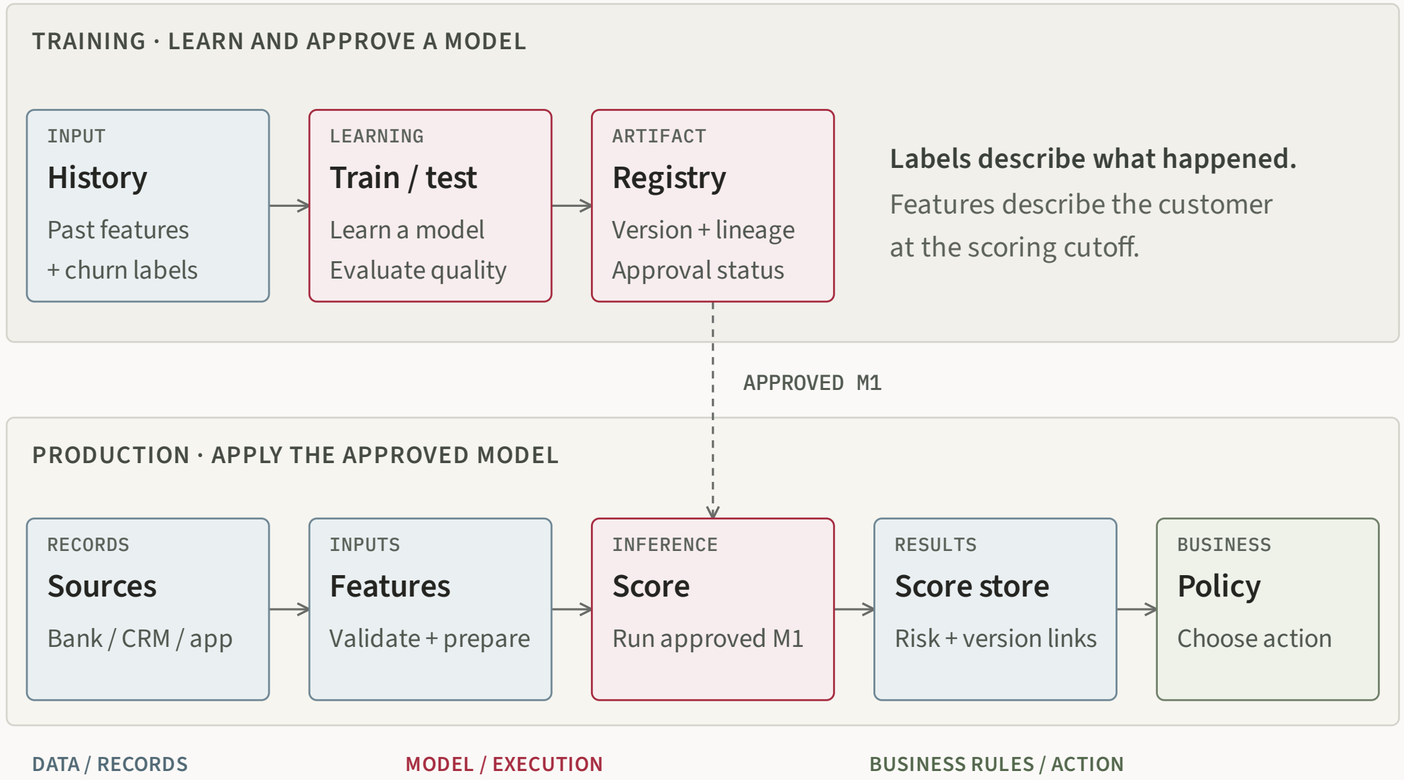
Tools make evidence easier to collect; architecture defines what that evidence means.

BANK EXAMPLE A missing CRM feature needs a contract-failure response; a changed valid population needs a different investigation.

BLOCK 2 / PRACTICAL TOOLS

ML architecture connects five responsibilities

Return to the full system and the choices that make a prediction usable.



Key concepts

- **Data:** preserve meaning, validity, and temporal consistency.
- **Models:** evaluate uncertainty and manage compatible approved artifacts.
- **Operations:** observe, diagnose, release, and recover.
- **Decisions:** protect access and govern appropriate business action.

Day 2: foundation models and agents retain these control needs, while components and failure modes change.

BANK EXAMPLE Ana’s score becomes usable through validated inputs, traceable execution, an approved release, and a separate campaign policy.

BLOCK 2 / PRACTICAL TOOLS

Sources and adaptation boundaries

The bibliography supplies concepts; the bank diagrams and incidents are explicit teaching applications.

Source	Role
Kalinowski, Ch. 13–14	Architecture, serving, design principles and patterns
Bass, Engineering AI Systems	Bank churn; security, privacy and observability
Nazir et al., JSS 2024	Challenges, practices and contextual design decisions
Bucaioni et al., JSS 2026	Quality checklist; published Table 1 spot-check
Breck / NIST supplements	Testing, release / recovery, threat and privacy framing

Key concepts

- **Core readings:** architectural decisions, serving, maintenance, and quality concerns.
- **Supporting readings** extend monitoring, governance, and security.
- **Illustrations:** Ana, scores, dates, batches, and incidents are invented.
- **Source limit:** the separately catalogued checklist corrigendum remains unread.

The separate guide records reading anchors and adaptation limits.

BANK EXAMPLE No new churn outcome, repaired score, retention benefit, or real bank tool execution is claimed.